Mise en production d'un système IA

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Introduction au déploiement de modèles **3**

II - Créer une interface avec Streamlit **5**

III - Packaging, reproductibilité et maintenance **39**

IV - L'essentiel **40**

V - Synthèse **41**

VI - Auto-évaluation **44**

A. Test..... 44

Solutions des exercices **47**

Tous droits réservés à STUDI - Reproduction interdite

I Introduction au déploiement de modèles

Une fois que nous avons entraîné un modèle de Machine Learning et que nous avons un pipeline ETL, le véritable défi commence. Le mettre à disposition d'utilisateurs réels.

Un modèle, aussi performant soit-il en laboratoire, ne sert à rien s'il n'est pas accessible, compréhensible et maintenu dans le temps.

Nous allons prendre un dataset qui illustre parfaitement cette problématique. Airbnb à Paris. Il contient les données des logements disponibles à Paris, avec leurs prix, emplacements et caractéristiques. Permettant de construire un modèle de prédiction du prix d'une annonce. Cependant, pour que ce modèle soit utile, il doit être intégré dans une interface claire et interactive.

Le déploiement implique plusieurs aspects clés.

La traçabilité

Il faut assurer la traçabilité, savoir avec quel jeu de données et quels paramètres un modèle a été entraîné. Cela permet de comparer différentes versions et de détecter si un modèle devient obsolète à cause d'un changement dans les données, par exemple lors d'un événement majeur comme les Jeux Olympiques qui bouleversent les prix de l'hébergement, une finale d'un événement sportif, un concert d'un artiste mondialement connu, etc. C'est pour ces raisons qu'il faut prévoir la surveillance du modèle en production, un monitoring peut mesurer régulièrement l'écart entre les prix prédits et les prix réellement observés, afin de déclencher un réentraînement quand l'erreur dépasse un certain seuil.

Pour rendre ce modèle utilisable, nous allons utiliser Streamlit. Cet outil permet de créer une application web en Python sans compétences particulières en développement frontend. Dans notre future application Airbnb, nous pouvons afficher une carte interactive de Paris avec les logements existants, puis proposer un formulaire dans lequel l'utilisateur renseigne les caractéristiques de son logement fictif. Une fois ces données saisies, un clic sur le bouton « *prédire* » lance le modèle et retourne une estimation du prix par nuit.

La reproductibilité et la maintenance

Cependant le déploiement ne s'arrête pas à la mise en ligne d'une application. La question de la reproductibilité et de la maintenance est centrale. Sans versioning, il est impossible de savoir quel modèle tourne actuellement en production. Une bonne pratique est d'utiliser des outils comme Mlflow ou DVC pour enregistrer les versions des modèles, les données utilisées et les performances obtenues. Cela signifie une trace des modèles entraînés avec les données de 2023 et de 2024 afin de comparer leur pertinence. En parallèle, il est crucial d'intégrer des tests simples dans le cycle de vie du projet. Par exemple, on peut s'assurer qu'un modèle ne prédit jamais un prix négatif, ou un prix qui pourrait entraîner des pertes financières pour le logeur. En s'assurant que le modèle reste dans un intervalle cohérent avec le marché. Enfin la documentation joue un rôle souvent sous-estimé, elle permet de comprendre, plusieurs mois plus tard, pourquoi certains choix ont été faits, comme l'exclusion d'un quartier ou l'utilisation d'une méthode d'encodage particulière.

Remarque

En conclusion, déployer un modèle de Machine Learning ou Deep Learning n'est pas seulement créer une interface avec Streamlit. C'est créer un projet avec une valeur dans une certaine rigueur afin de conduire à son utilisation et reproduction de tous.

Tous droits réservés à STUDI - Reproduction interdite

Avant toute interface, commençons par comprendre et préparons les données. L'idée va être de charger le dataset Airbnb Paris depuis Inside Airbnb. Ensuite, de nettoyer les données et explorer les prix par quartier et type de logement. Entraîner un modèle de prédiction simple et le sauvegarder pour la suite pour la partie Streamlit.

Tous droits réservés à STUDI - Reproduction interdite

II Créer une interface avec Streamlit

Az Définition

Streamlit est un framework open-source en Python. Conçu pour rapidement créer des applications web interactives à partir de scripts de Data Science et de Machine Learning, il est particulièrement apprécié dans l'écosystème Python car il est très accessible et évite d'avoir à écrire du code front-end.

Crée en 2019 par Adrien Treuille, Amanda Kelly et Thiago Teixeira, l'idée derrière Streamlit était de démocratiser la Data Science en éliminant de la complexité au niveau du développement web. Streamlit sera racheté en 2022 par Snowflake, géant du cloud data pour environs 800 millions de dollars. Aujourd'hui Streamlit reste très utilisé pour la visualisation de données et le partage de projets. Une grande communauté continue d'ajouter des composants et des fonctionnalités.

Pour installer Streamlit, il convient d'avoir Python installé en version supérieure à 3.8.

Voici la commande pour installer Streamlit :

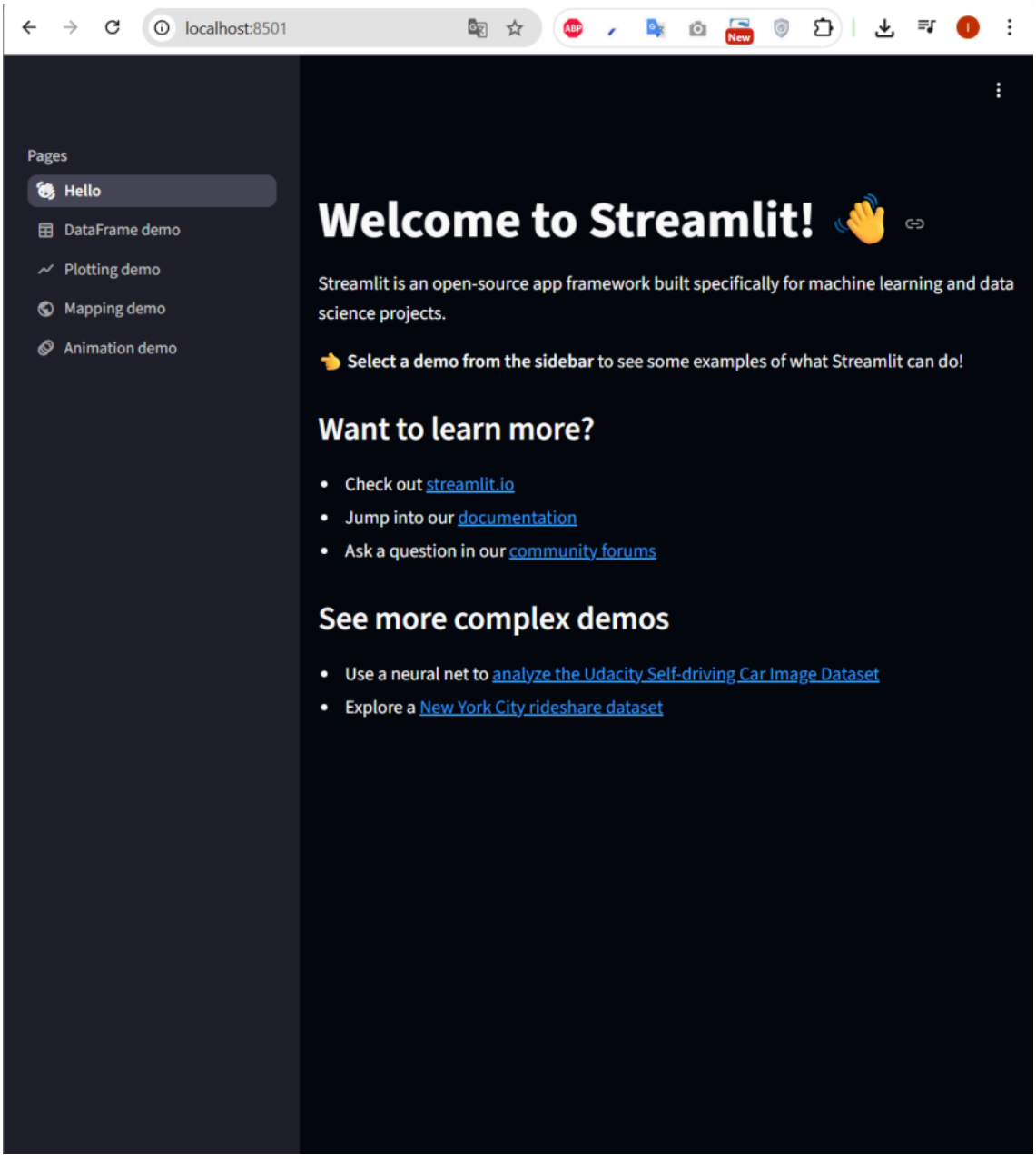
```
pip install streamlit
```

Pour vérifier que l'installation s'est bien déroulée, voici la commande utile :

```
streamlit hello
```

Vous devriez avoir l'écran suivant, avec une page de votre navigateur par défaut qui s'ouvre en local à : http://localhost:8501/

Tous droits réservés à STUDI - Reproduction interdite



Comme vous pouvez le constater en naviguant à travers les onglets, il y a énormément de possibilités qui s'offrent à nous et d'idées de projets que vous devez avoir envie de réaliser avec toutes les compétences et techniques que vous avez acquis jusqu'à présent.

Créons maintenant notre premier projet Streamlit depuis 0.

Créons un fichier app.py

Voici la structure de base de Streamlit :

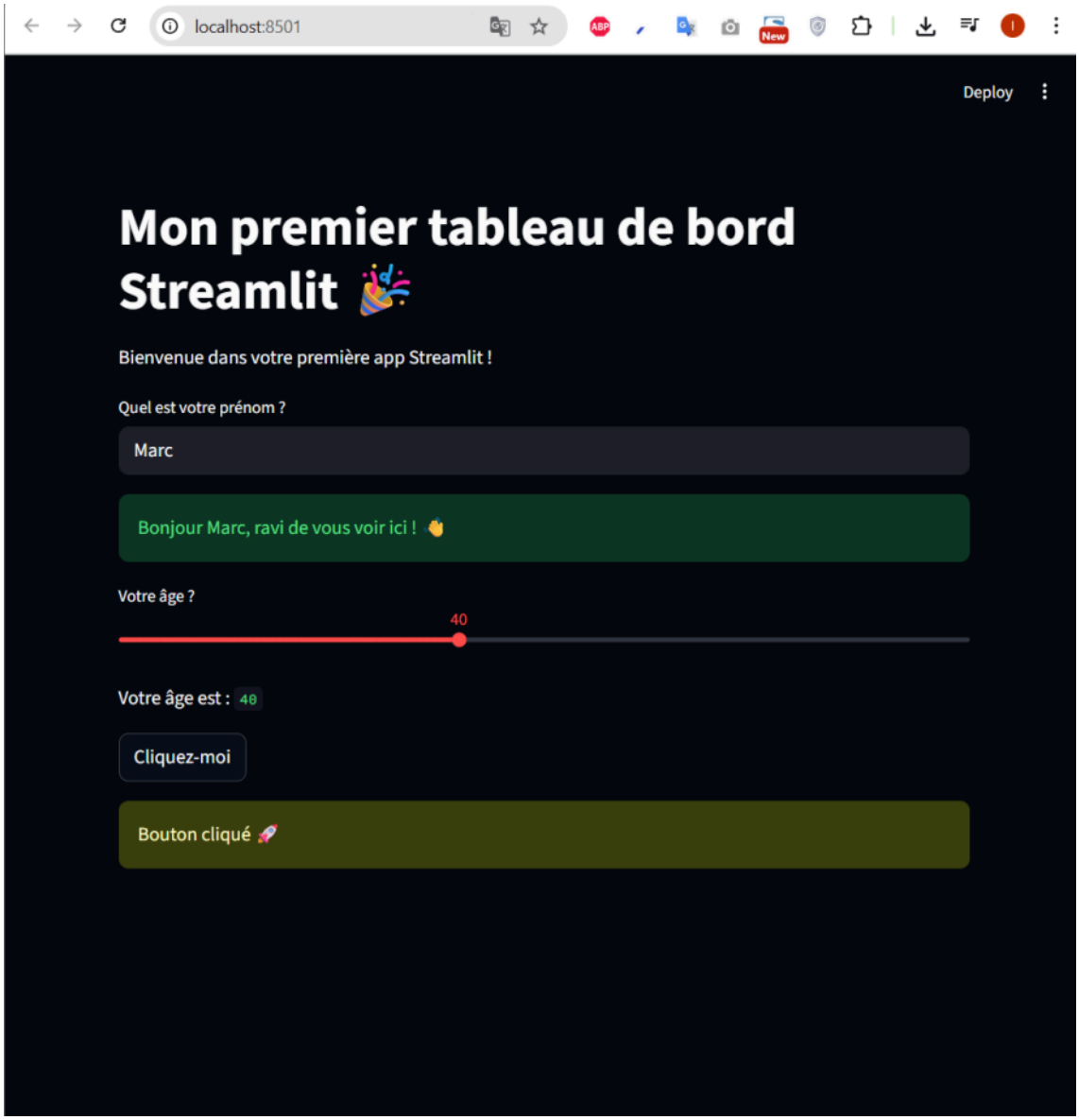
```
1 import streamlit as st
2
3 # Titre
4 st.title("Mon premier tableau de bord Streamlit 🎉")
5
6 # Texte
7 st.write("Bienvenue dans votre première app Streamlit !")
8
9 # Entrée utilisateur
10 nom = st.text_input("Quel est votre prénom ?")
11
12 if nom:
13     st.success(f"Bonjour {nom}, ravi de vous voir ici ! 👋")
14
```


Tous droits réservés à STUDI - Reproduction interdite

```
15 # Slider
16 age = st.slider("Votre âge ?", 0, 100, 25)
17 st.write("Votre âge est :", age)
18
19 # Bouton
20 if st.button("Cliquez-moi"):
21     st.warning("Bouton cliqué 🚀")
```

Pour lancer notre application dans le terminal, voici la commande :

```
1 streamlit run app.py
```



En effet, c'est un peu trivial comme application, mais vous pouvez la partager si besoin avec les URL mentionnées dans votre terminal.

Nous pouvons intégrer facilement Matplotlib, Seaborn ou Pandas.

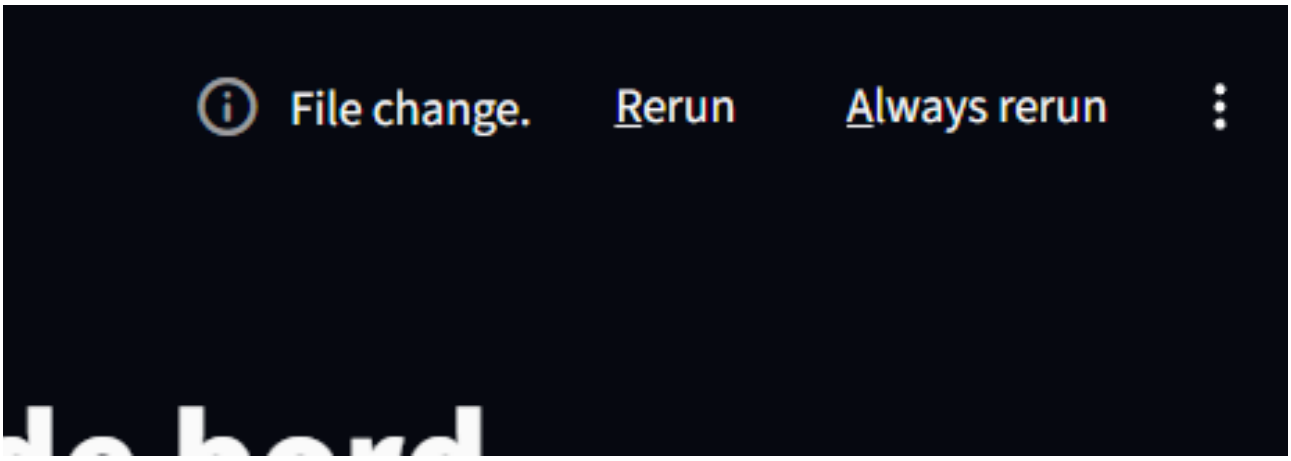
Dessinons un histogramme assez simple :

```
1 import streamlit as st
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # Titre
7 st.title("Mon premier tableau de bord Streamlit 🎉")
8
9 # Texte
```

Tous droits réservés à STUDI - Reproduction interdite

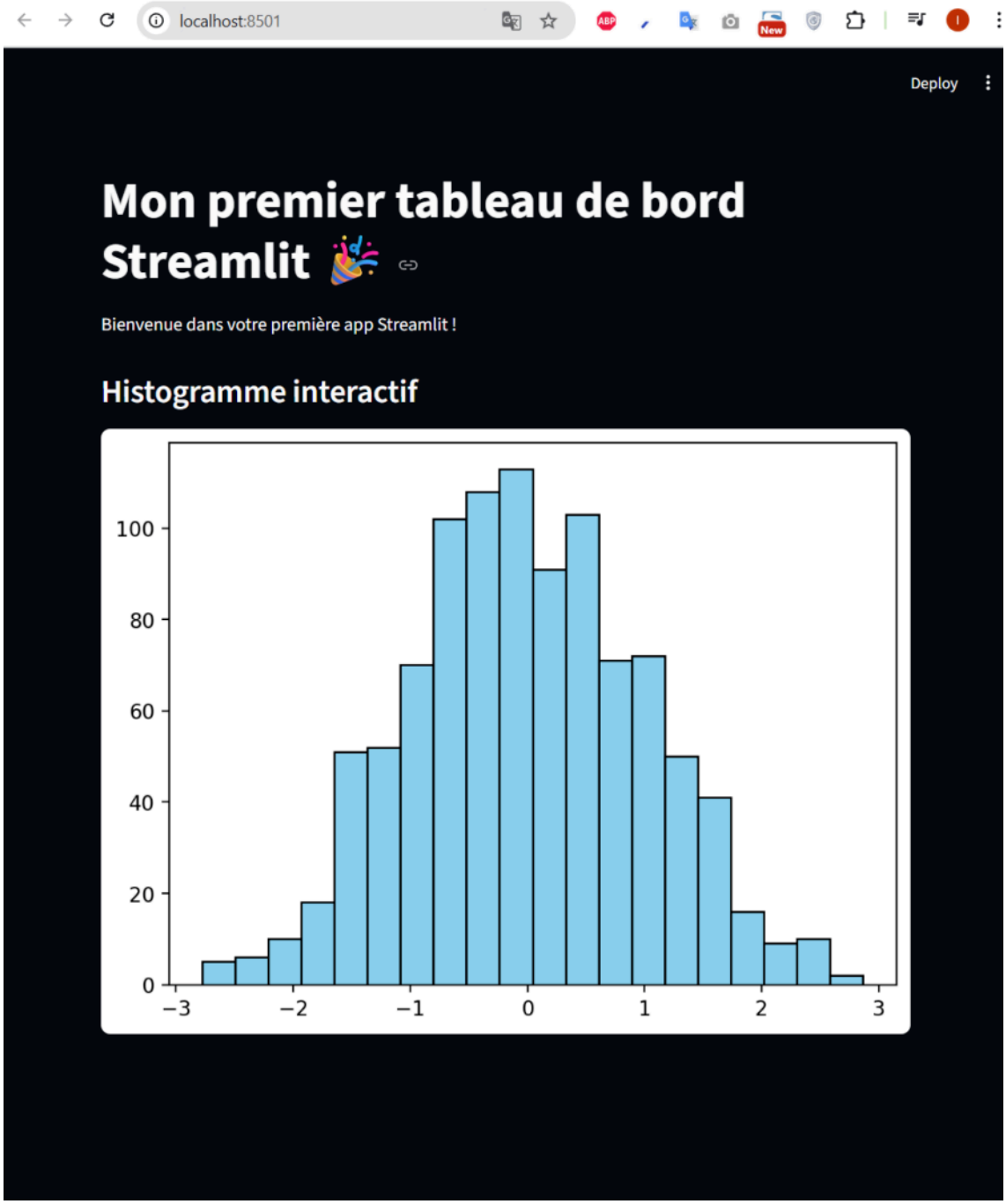
```
st.write("Bienvenuae dans votre première app Streamlit !")

# Jeu de données aléatoire
data = pd.DataFrame({
    "x": np.random.randn(1000)
})

st.subheader("Histogramme interactif")
fig, ax = plt.subplots()
ax.hist(data["x"], bins=20, color="skyblue", edgecolor="black")
st.pyplot(fig)
```

Notez que dès lors que votre script app.py est modifié, vous pouvez faire un Rerun pour actualiser votre page, en haut à droite de votre page Streamlit et charger les nouvelles données.



Tous droits réservés à STUDI - Reproduction interdite

Voici notre nouvelle page :



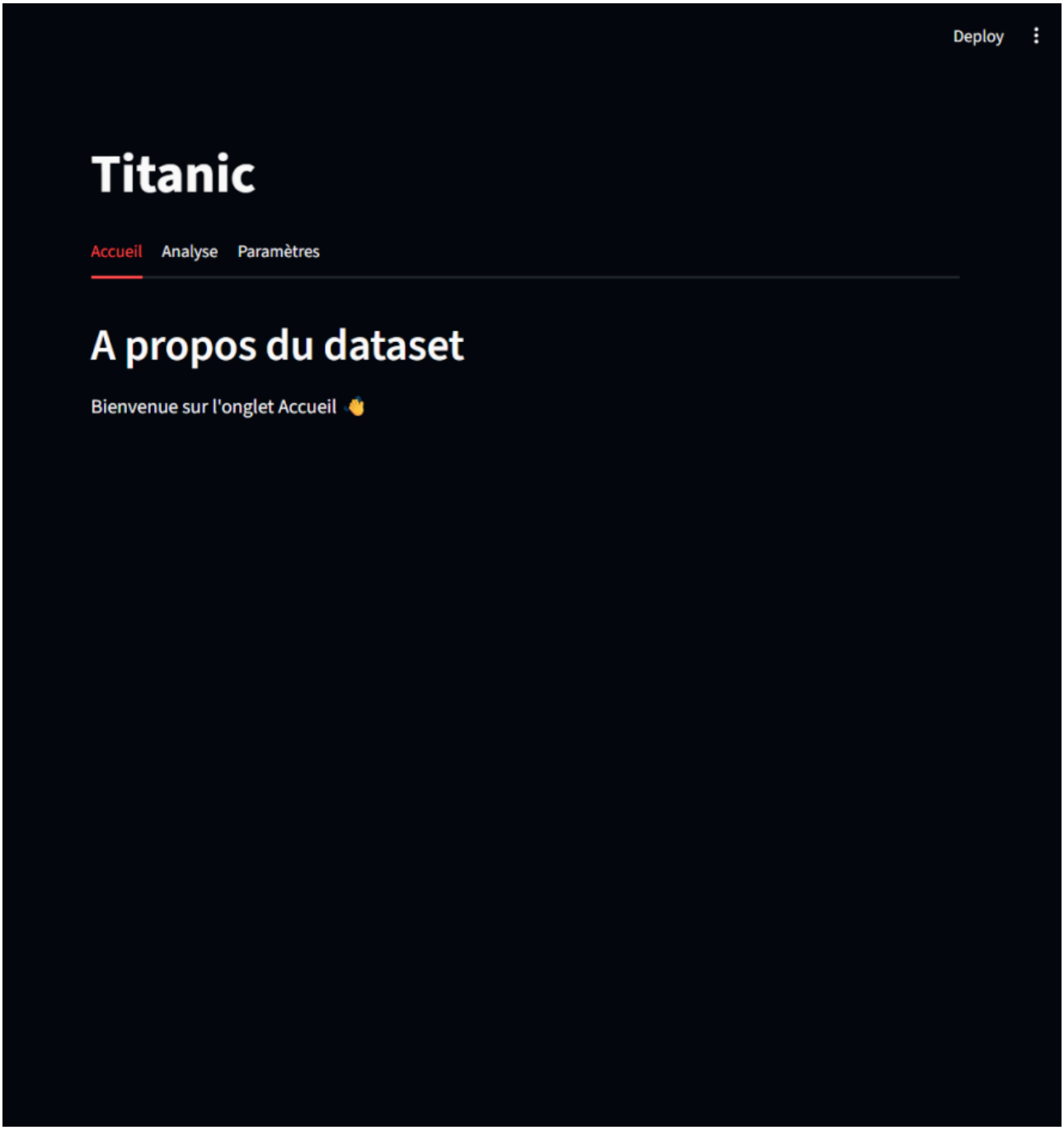
Partons sur un projet un peu plus « *évolué* » et surtout ludique. Nous allons créer une interface qui va reprendre les caractéristiques des passagers du Titanic et entraîner sur le dataset un modèle de classification Random Forest.

Faisons une analyse exploratoire du dataset, gardons cette bonne pratique avant de travailler sur notre modèle.

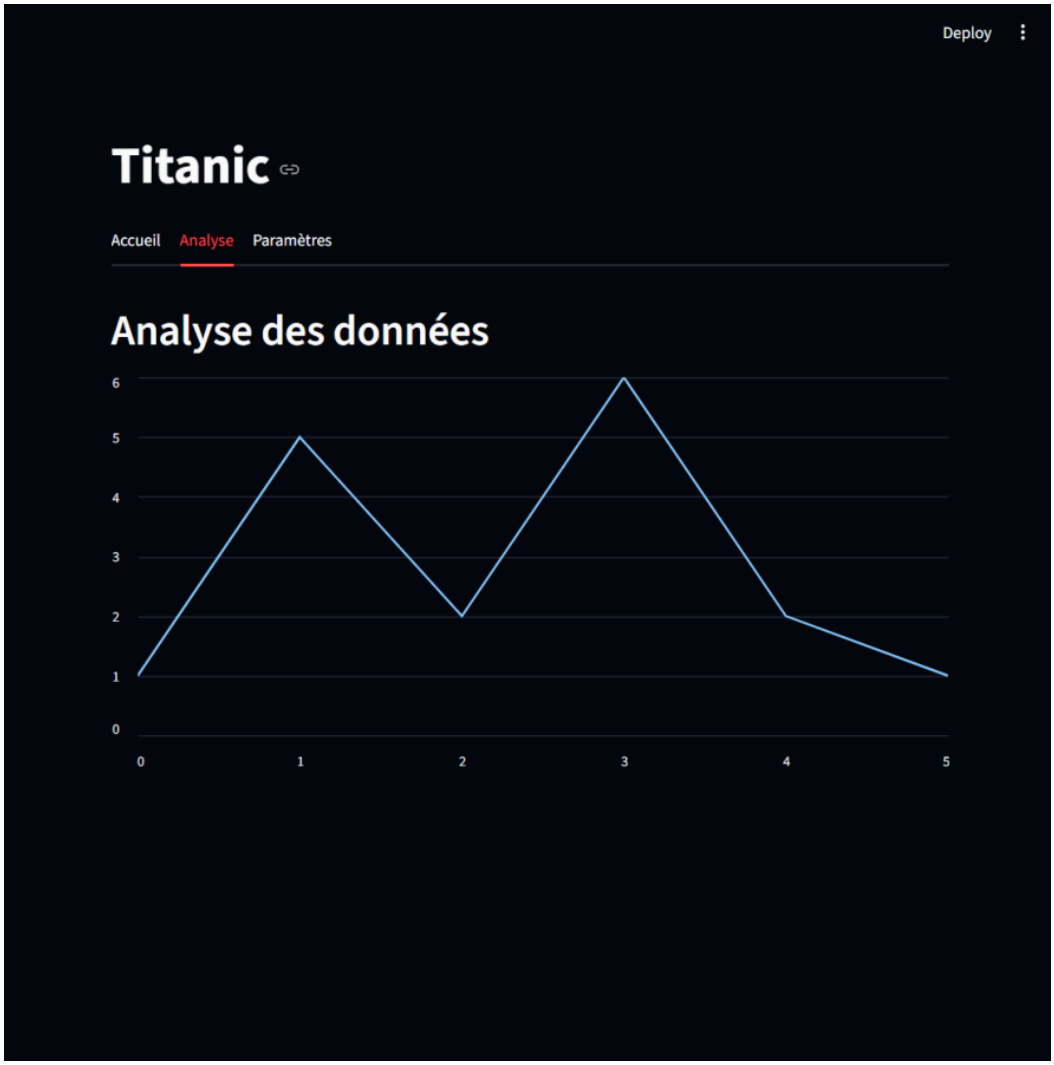
Pour cela nous allons structurer notre app en 3 onglets de la manière suivante :

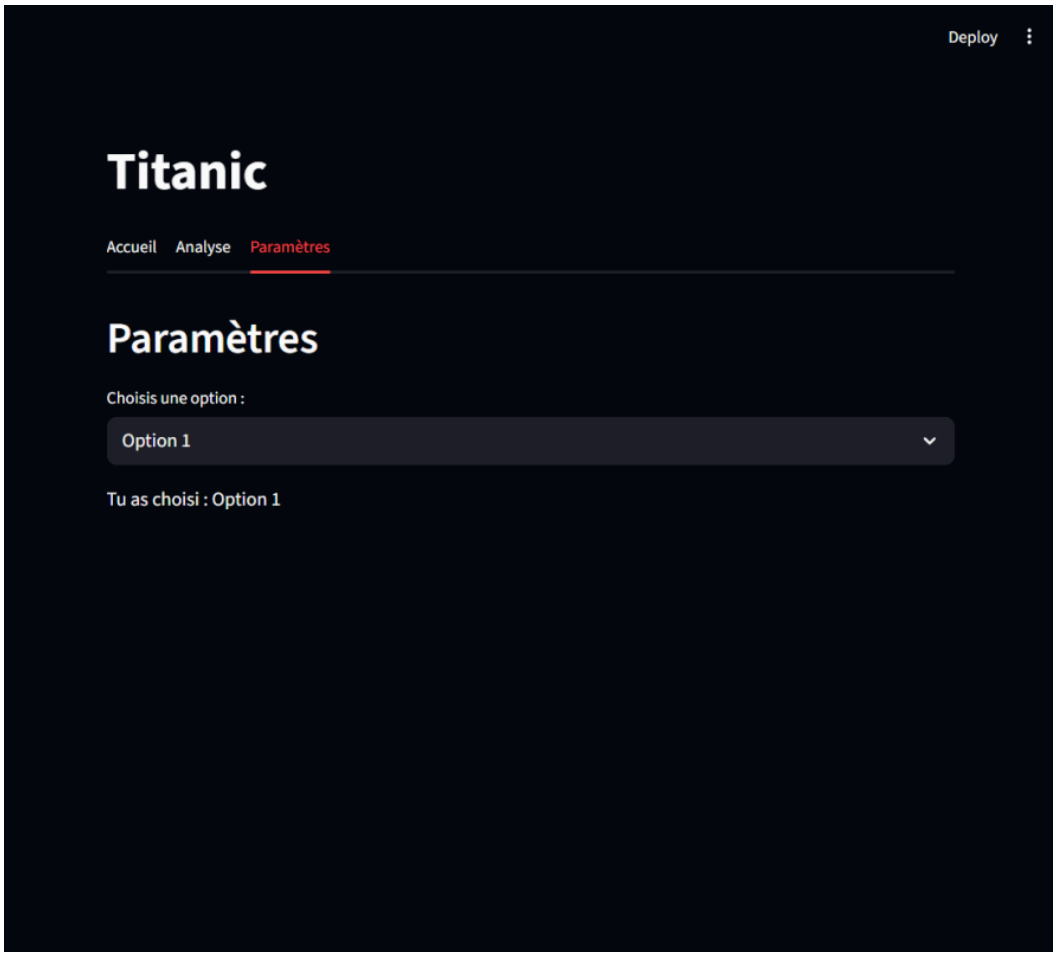
```
st.title("Titanic")

# Création de 3 onglets
onglet1, onglet2, onglet3 = st.tabs(["Accueil", "Analyse", "Paramètres"])

with onglet1:
    st.header("A propos du dataset")
    st.write("Bienvenue sur l'onglet Accueil 👋")

```

```
10 with onglet2:
11     st.header("Analyse des données")
12     st.line_chart({"data": [1, 5, 2, 6, 2, 1]})
13
14 with onglet3:
15     st.header("Paramètres")
16     choix = st.selectbox("Choisis une option :", ["Option 1", "Option 2", "Option
3"])
17     st.write("Tu as choisi :", choix)
```



Tous droits réservés à STUDI - Reproduction interdite



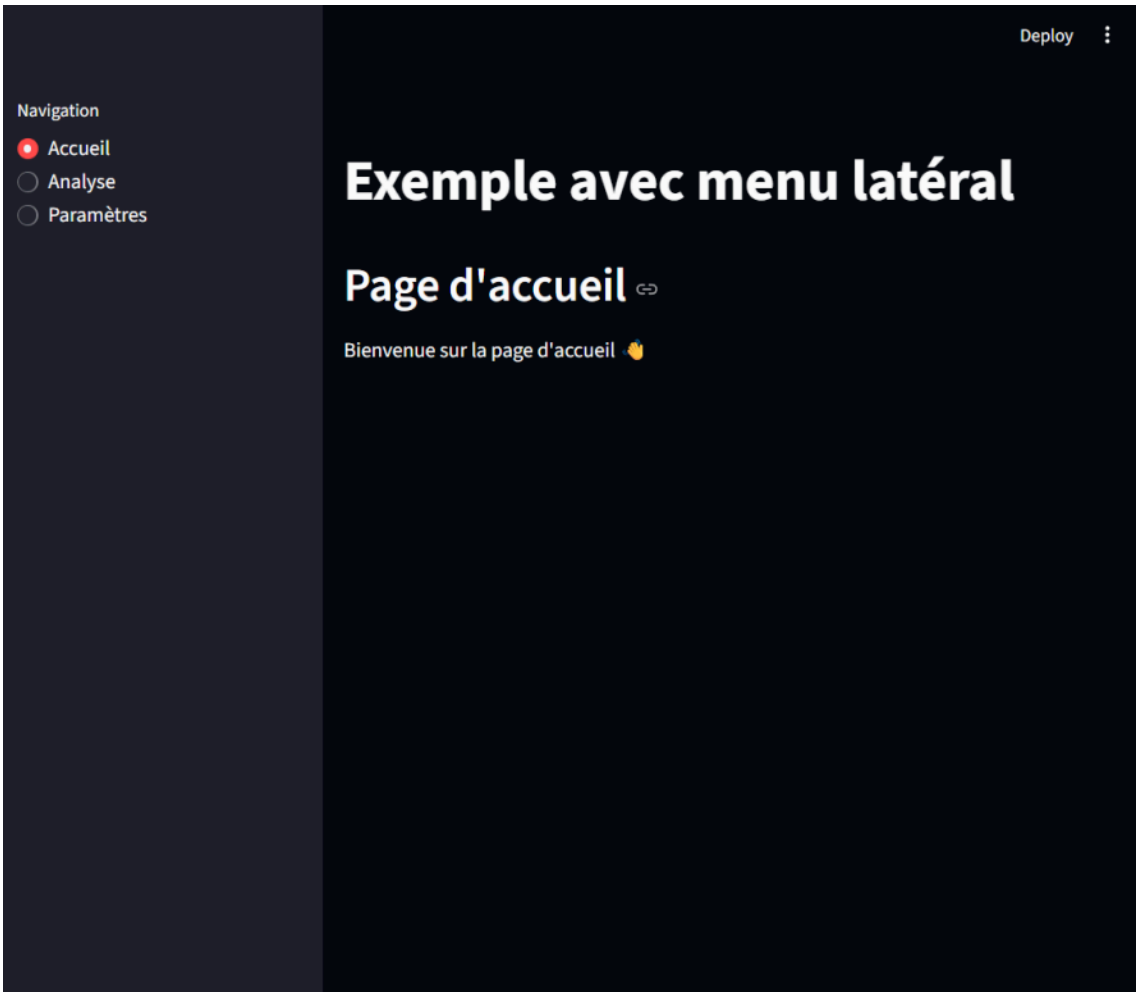


Tous droits réservés à STUDI - Reproduction interdite

Le but sera de créer un projet complet en lien avec le dataset du titanic.

Nous pouvons aussi opter pour une approche en sidebar, testez et voyez par vous-même la différence :

```
import streamlit as st

st.title("Exemple avec menu latéral")

# Menu dans la sidebar
menu = st.sidebar.radio(
    "Navigation",
    ["Accueil", "Analyse", "Paramètres"]
)

# Affichage du contenu selon le choix
if menu == "Accueil":
    st.header("Page d'accueil")
    st.write("Bienvenue sur la page d'accueil 👋")

elif menu == "Analyse":
    st.header("Analyse des données")
    st.line_chart({"data": [1, 5, 2, 6, 2, 1]})

elif menu == "Paramètres":
    st.header("Paramètres")
    choix = st.selectbox("Choisis une option :", ["Option 1", "Option 2", "Option 3"])
    st.write("Tu as choisi :", choix)
```



Tous droits réservés à STUDI - Reproduction interdite

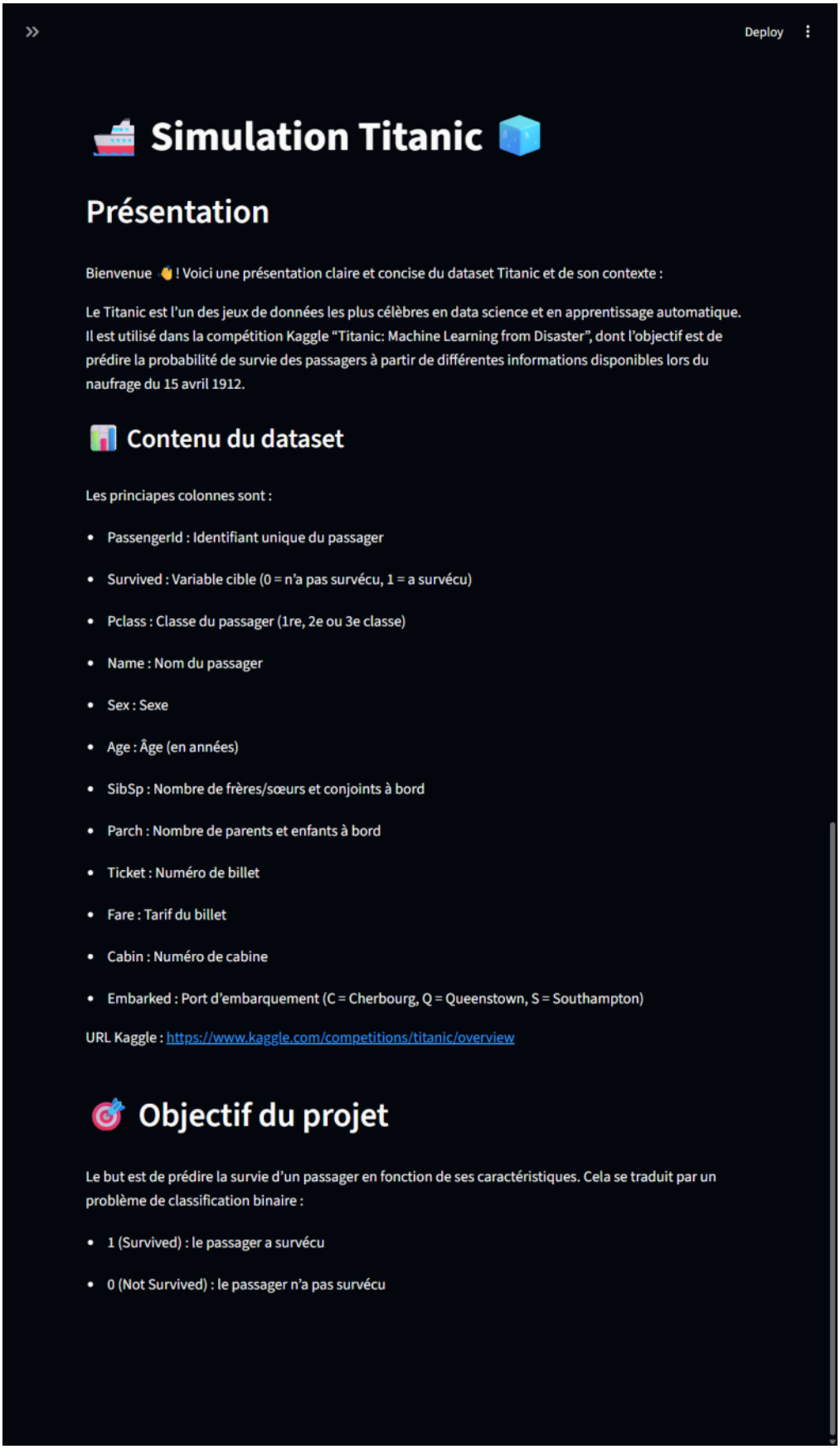
Voici un exemple des deux méthodes combinées :

```python
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


st.title("Exemple combiné : Sidebar + Onglets")

# --- Menu latéral ---
section = st.sidebar.radio(
    "Navigation",
    ["Accueil", "Analyse", "Paramètres"]
)

# --- Contenu selon la section choisie ---
if section == "Accueil":
    st.header("Page d'accueil")
    st.write("Bienvenue 👋, choisis une section dans la sidebar.")

elif section == "Analyse":
    st.header("Analyse des données")

    # Onglets pour organiser l'analyse
    onglet1, onglet2 = st.tabs(["Graphiques", "Tableau"])

    with onglet1:
        st.subheader("Graphiques")
        st.line_chart({"données": [1, 5, 2, 6, 3, 8]})

    with onglet2:
        st.subheader("Tableau")
        st.dataframe({
            "Nom": ["Alice", "Bob", "Charlie"],
            "Score": [85, 90, 78]
        })

elif section == "Paramètres":
    st.header("Paramètres de l'application")

    # Onglets pour organiser les réglages
    onglet1, onglet2 = st.tabs(["Options", "Avancé"])

    with onglet1:
        st.write("Réglages simples")
        choix = st.selectbox("Choisis une option :", ["Option 1", "Option 2", "Option 3"])
        st.write("Tu as choisi :", choix)

    with onglet2:
        st.write("Réglages avancés")
        valeur = st.slider("Choisis une valeur :", 0, 100, 50)
        st.write("Valeur sélectionnée :", valeur)
```

Tous droits réservés à STUDI - Reproduction interdite

Faisons maintenant un peu de mise en forme pour rendre attrayante notre application.

Tous droits réservés à STUDI - Reproduction interdite



Voici le code associé si vous souhaitez la même structure de départ :

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


```

Tous droits réservés à STUDI - Reproduction interdite

```python
st.title(" 🚢 Simulation Titanic 🧊 ")

# --- Menu latéral ---
section = st.sidebar.radio(
    "Pages",
    ["Accueil", "Analyse", "Paramètres"]
)

# ------------------ " Page 1 " ------------------

# --- Contenu selon la section choisie ---
if section == "Accueil":
    st.header("Présentation")
    st.write('')
    st.write("Bienvenue 👋! Voici une présentation claire et concise du dataset Titanic et de son contexte :")
    st.write("Le Titanic est l'un des jeux de données les plus célèbres en data science et en apprentissage automatique. Il est utilisé dans la compétition Kaggle “Titanic: Machine Learning from Disaster”, dont l'objectif est de prédire la probabilité de survie des passagers à partir de différentes informations disponibles lors du naufrage du 15 avril 1912.")
    st.subheader("📊 Contenu du dataset")
    st.write('')
    st.write("Les princiapes colonnes sont : ")
    st.write(" - PassengerId : Identifiant unique du passager")
    st.write("- Survived : Variable cible (0 = n'a pas survécu, 1 = a survécu) ")
    st.write("- Pclass : Classe du passager (1re, 2e ou 3e classe) ")
    st.write("- Name : Nom du passager")
    st.write("- Sex : Sexe")
    st.write("- Age : Âge (en années)")
    st.write("- SibSp : Nombre de frères/sœurs et conjoints à bord")
    st.write("- Parch : Nombre de parents et enfants à bord")
    st.write("- Ticket : Numéro de billet")
    st.write("-  Fare : Tarif du billet")
    st.write("- Cabin : Numéro de cabine")
    st.write("- Embarked : Port d'embarquement (C = Cherbourg, Q = Queenstown, S = Southampton)")
    st.write(" URL Kaggle : https://www.kaggle.com/competitions/titanic/overview")
    st.write('')
    st.header("🎯 Objectif du projet")
    st.write('')
    st.write('Le but est de prédire la survie d'un passager en fonction de ses caractéristiques. Cela se traduit par un problème de classification binaire :')
    st.write('- 1 (Survived) : le passager a survécu')
    st.write('- 0 (Not Survived) : le passager n'a pas survécu')

# ------------------ " Page 2 " ------------------

elif section == "Analyse":
    st.header("Analyse exploratoire des données")

    # Onglets pour organiser l'analyse
    onglet1, onglet2 = st.tabs(["Graphiques", "Tableau"])

    with onglet1:
        st.subheader("Graphiques")
        st.line_chart({"données": [1, 5, 2, 6, 3, 8]})

    with onglet2:
        st.subheader("Tableau")
        st.dataframe({
            "Nom": ["Alice", "Bob", "Charlie"],
            "Score": [85, 90, 78]
        })
```


Tous droits réservés à STUDI - Reproduction interdite

```python

# ----------------- " Page 3 " -----------------

elif section == "Paramètres":
    st.header("Paramètres de l'application")

    # Onglets pour organiser les réglages
    onglet1, onglet2 = st.tabs(["Options", "Avancé"])

    with onglet1:
        st.write("Réglages simples")
        choix = st.selectbox("Choisis une option :", ["Option 1", "Option 2", "Option 3"])
        st.write("Tu as choisi :", choix)

    with onglet2:
        st.write("Réglages avancés")
        valeur = st.slider("Choisis une valeur :", 0, 100, 50)
        st.write("Valeur sélectionnée :", valeur)
```

Si vous voulez intégrer du code dans votre application, il est recommandable d'utiliser la méthode markdown de la manière suivante :

```python
st.write("Chargeons notre dataset depuis une url :")
        st.markdown("""```python
        url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
        df = pd.read_csv(url)
                    """)
```


```python
st.write("Chargeons notre dataset depuis une url :")
st.markdown("""```python
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.
df = pd.read_csv(url)
""")
```


Tous droits réservés à STUDI - Reproduction interdite

Qui donnera le résultat suivant sur Streamlit :

Chargeons notre dataset :

```
# Charger depuis une URL Kaggle (copie publique)
        url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
        df = pd.read_csv(url)
```

Affichons les dimensions du dataset :

```
# Dimensions
st.subheader("Dimensions du dataset")
st.write(f"{df.shape[0]} lignes × {df.shape[1]} colonnes")
```

Faisons un aperçu des données :

```
    # Aperçu des 5 premières lignes
        st.subheader("Aperçu des données")
        st.dataframe(df.head())
```


Tous droits réservés à STUDI - Reproduction interdite

Dimensions du dataset

891 lignes × 12 colonnes

Aperçu des données

	PassengerId	Survived	Pclass	Name	Sex	Age	S
0	1	0	3	Braund, Mr. Owen Harris	male	22	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	
2	3	1	3	Heikkinen, Miss. Laina	female	26	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	
4	5	0	3	Allen, Mr. William Henry	male	35	

Affichons maintenant le type des colonnes :

```
# Types des colonnes
st.subheader("Types de colonnes")
st.code(df.dtypes.to_string())
```

Types de colonnes

```
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
```

Passons à l'étape 2, qui concerne la structure et qualité des données.

Pour cela nous allons écrire le code associé toujours sur la même page mais dans un nouvel onglet, à la suite de notre code du premier onglet. Voici la structure à suivre :

```
    with onglet2:
        st.header("Etape 2 : Structure et qualité des données ")
        st.subheader("🎯 Objectifs")
        st.write("- Identifier les colonnes avec des valeurs manquantes (NA).")
        st.write("- Vérifier s'il existe des doublons.")
        st.write("- Voir la cardinalité (nombre de valeurs uniques par colonne).")
        st.write("- Avoir une vision claire de la qualité globale des données.")
```

Tous droits réservés à STUDI - Reproduction interdite

```
        # Types des colonnes
        st.subheader("Types de colonnes")
        st.code(df.dtypes.to_string())
        st.write("")
        st.subheader("📝 Explications")
        st.write("- pd.read_csv(url) : charge directement les données depuis une URL p
        st.write("- df.shape : donne les dimensions du dataset.")
        st.write("- df.head() : montre les 5 premières lignes pour comprendre la struc
        st.write("- df.dtypes : permet de voir si les types sont cohérents (Age doit ê



    with onglet2:
        st.header("Etape 2 : Structure et qualité des données ")
        st.subheader("🎯 Objectifs")
        st.write("- Identifier les colonnes avec des valeurs manquantes (NA).")
        st.write("- Vérifier s'il existe des doublons.")
        st.write("- Voir la cardinalité (nombre de valeurs uniques par colonne).")
        st.write("- Avoir une vision claire de la qualité globale des données.")
```

Vérifions maintenant les valeurs manquantes :

```
# Vérification des valeurs manquantes
        st.subheader("Valeurs manquantes (%)")
        na_percent = df.isna().mean().round(3) * 100
        st.bar_chart(na_percent)
```

Ce qui donne le résultat suivant dans notre application :

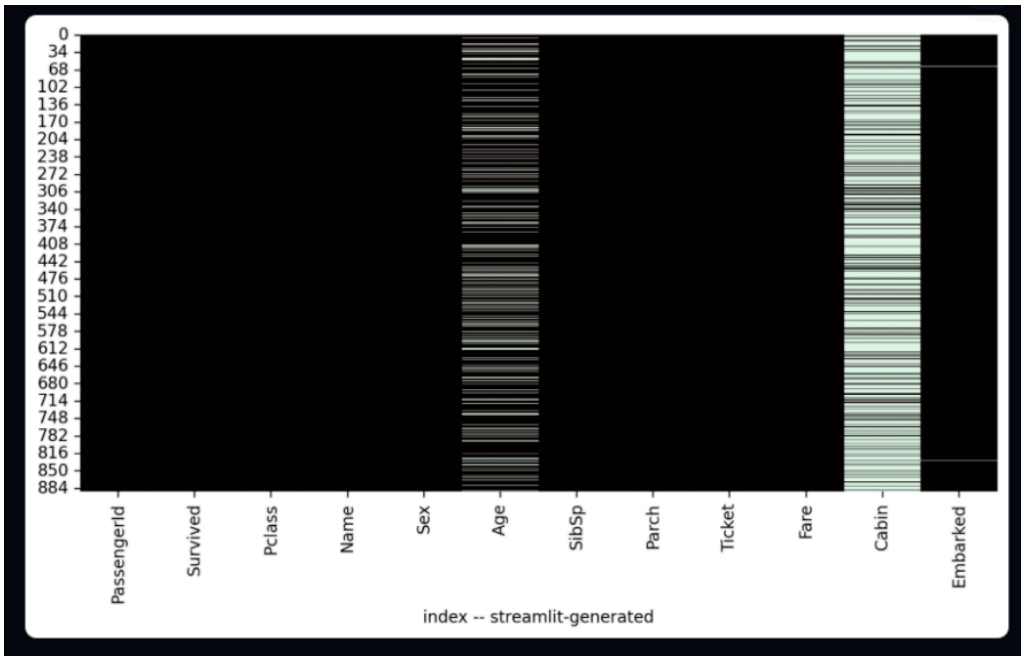
Faisons une petite analyse de ce que montre notre graphique. Il y a à peu près 20 % de valeurs manquantes pour l'Age. Certains passagers n'ont pas d'âge renseigné. Pour un modèle nous devrons soit imputer, faire une médiane par classe/sex soit créer une catégorie Age manquant.

Pour ce qui est des cabines, il y a peu près 77 % de valeurs manquantes : la majorité des cabines n'ont pas été notées. Ici la suppression ne posera pas de problèmes.

Embarked, est relativement faible entre 0 et 1 %, quelques passagers sans port d'embarquement. Ce qui est facile à imputer à une autre classe en S, Southampton est le plus fréquent. A noter qu'il n'y a pas de doublons.

Traçons une heatmap des valeurs manquantes :

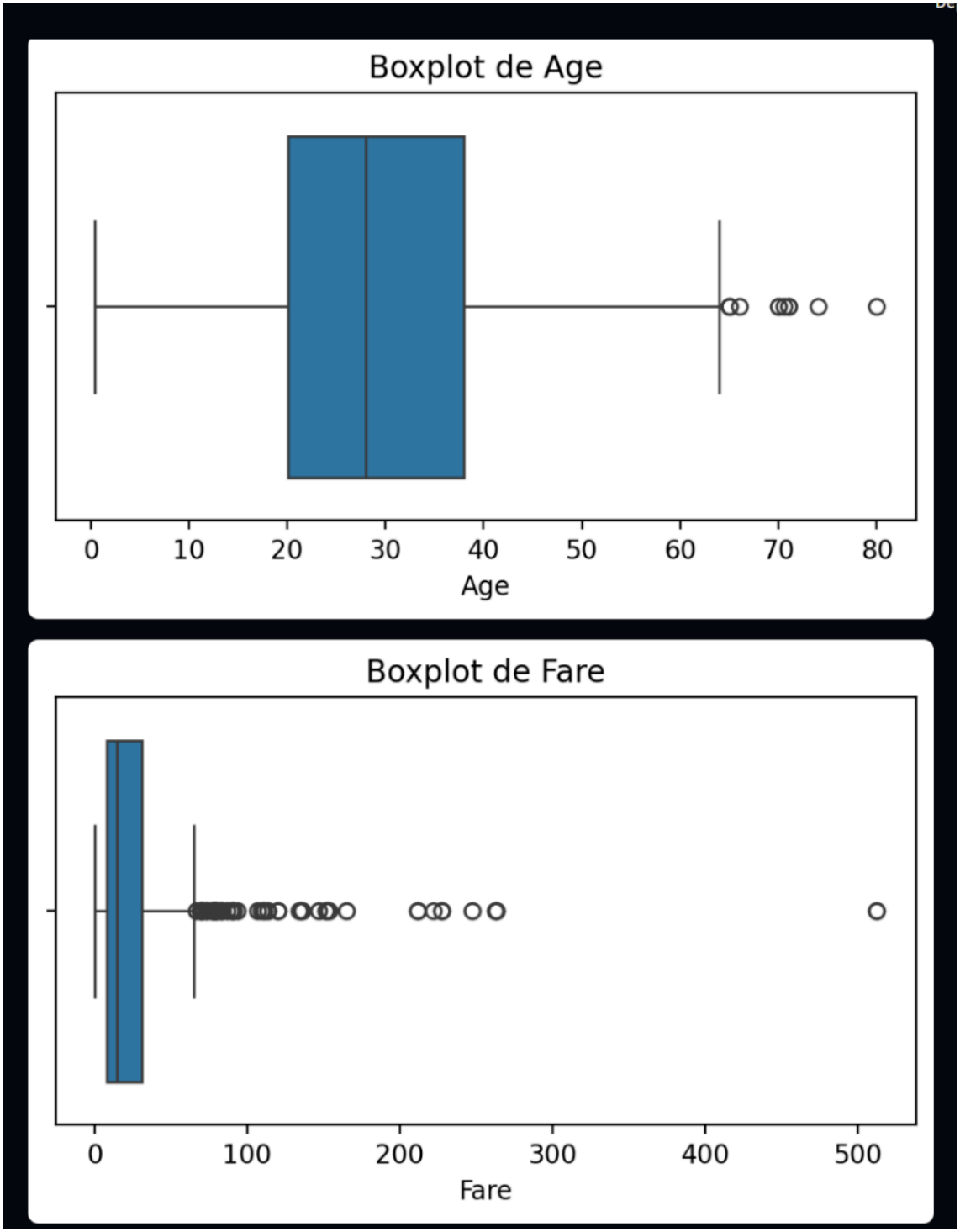
```
st.subheader("Heatmap des valeurs manquantes")
fig, ax = plt.subplots(figsize=(10,5))
sns.heatmap(df.isnull(), cbar=False, cmap="mako", ax=ax)
st.pyplot(fig)
```



Tous droits réservés à STUDI - Reproduction interdite

Ajoutons deux boxplots, l'un pour l'Age et l'autre pour les Fare (prix du billet), afin de détecter les valeurs aberrantes.

```
# Boxplots pour détecter les valeurs aberrantes
        st.subheader("Boxplots des variables numériques")

        num_cols = ["Age", "Fare"]
        for col in num_cols:
            fig, ax = plt.subplots(figsize=(6,3))
            sns.boxplot(x=df[col], ax=ax)
            ax.set_title(f"Boxplot de {col}")
            st.pyplot(fig)
```



Nous pouvons lire sur les graphiques que la plupart des passagers ont un âge entre 20 et 40 ans ce qui correspond à la boite centrale blue. La médiane est autour de 28 ans.

Quelques passagers sont plus âgés (Age > 60 ans).

Tous droits réservés à STUDI - Reproduction interdite

La majorité des billets ont été payé moins de 50 livres, la médiane étant assez basse aux alentours de 15 livres. Beaucoup de passagers ont payé beaucoup plus cher mais restent tout de même minoritaire au-delà de 80 livres.

Si maintenant nous regardons la cardinalité des variables, voici ce qui peut être intéressant, Pclass, Survived, Age, Embarked, SibSp, Parch. Le reste contient soit beaucoup de données manquantes, ou soit un très grand nombre de valeurs uniques. Qui risquent de ne rien apporter au modèle.

Passons à l'étape 3 en nettoyant les valeurs manquantes (age cabin, embarked).

Nous allons les traiter, dans le dataset l'âge sera imputé par la médiane, par sexe et classe pour plus de pertinence, le port d'embarquement est complété par la modalité la plus fréquente et la cabine est simplifiée en une variable binaire indiquant si l'information est connue ou non.

Nous allons aussi faire du feature engineering pour enrichir le dataset de nouvelles variables. Comme la taille de la famille, ou le fait d'être seul ou accompagné. Ces informations permettront d'exploiter les signaux cachés dans les données brutes.

Nous encoderons par la suite les variables catégorielles (sexe, embarquement, titre, pont) pour les transformer en colonnes numériques grâce au one-hot encoding. Nous ferons aussi une normalisation sur les variables numériques comme le tarif pour limiter l'impact des valeurs extrêmes. Nous aurons un jeu de données final, propre et prêt à l'emploi.

Si nous regardons notre application les valeurs affichées de age et fare peuvent pour un non initié être bizarres, regardons de plus prêt.

✅ Jeu de données final (prêt pour l'entraînement)

	Survived	Pclass	Age	Fare	FamilySize	IsAlone	CabinKnown	Sex_male	Embarked_Q
0	0	3	-0.5349	-0.5024	2	0	0	1	0
1	1	1	0.6684	0.7868	2	0	1	0	0
2	1	3	-0.2341	-0.4889	1	1	0	0	0
3	1	1	0.4428	0.4207	2	0	1	0	0
4	0	3	0.4428	-0.4863	1	1	0	1	0
5	0	3	-0.3093	-0.4781	1	1	0	1	1
6	0	1	1.8717	0.3958	1	1	1	1	0
7	0	3	-2.039	-0.2241	5	0	0	1	0
8	1	3	-0.1589	-0.4243	3	0	0	0	0
9	1	2	-1.1365	-0.043	2	0	0	0	0

Comme expliqué plus haut, nous avons standardisé les valeurs. Un traitement de type StandardScaler de scikit-learn a été appliqué.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Prenons quelques exemples :

- Si Age = - 0.6684, le passager est plus jeune que la moyenne
- Si Age = 1.27, le passager est plus âgé que la moyenne
- Si Fare = - 0.5024, le billet payé est moins cher que la moyenne
- Si Fare = 0.7868, le billet payé est plus cher que la moyenne

Ces valeurs sont normalisées pour que les algorithmes de Machine Learning puissent comparer correctement les variables. Cela évitera que Fare écrase Age car étant sur une autre échelle.

Passons à l'étape 4, la conception du modèle de Machine Learning.

Nous allons garder la structure actuelle et créer un nouvel onglet sur la page analyse.

Quelques métriques seront implémentées avec différents algorithmes ainsi vous pourrez voir les différentes métriques associées aux différents algorithmes.

Rappel

Faisons un petit résumé des résultats :

- **Régression Logistique** : Accuracy ≈ 0.84 → très bon équilibre entre rappel et précision, modèle simple mais efficace.
- **Random Forest (optimisée)** : Accuracy ≈ 0.80 → performante, mais légèrement moins bonne que la régression logistique, malgré l'optimisation.
- **Gradient Boosting** : Accuracy ≈ 0.79 → comparable à la Random Forest, bonnes performances globales.
- **SVM** : Accuracy ≈ 0.61 → très mauvais rappel sur la classe « *survivants* » (le modèle prédit presque tout en « *non-survivant* »), donc inutilisable.
- **XGBoost** : Accuracy ≈ 0.80 → résultats proches de la Random Forest et du Gradient Boosting, bonne robustesse mais pas le meilleur modèle ici.

La Régression Logistique est paradoxalement le modèle qui donne les meilleurs résultats sur ce dataset, avec une précision globale de 84 % et un bon équilibre entre classes. Les modèles ensemblistes (RF, GB, XGBoost) atteignent environ 80 %, tandis que le SVM échoue totalement à reconnaître les survivants.

Pour conclure, la Régression Logistique fonctionne particulièrement bien sur le dataset Titanic parce que les variables explicatives (sexe, classe, âge, embarquement, etc.) sont relativement peu nombreuses et présentent des relations linéaires simples avec la variable cible (Survived). Par exemple être une femme ou voyager en première classe augmente directement les chances de survie, ce qui est bien capté par un modèle linéaire. Dans ce cas, la simplicité de la régression logistique devient un avantage, car elle évite de trop compliquer l'apprentissage et réduit les risques de surapprentissage. A l'inverse, des modèles plus complexes comme la Random Forest, le Gradient Boosting ou XGBoost cherchent à capter des relations non-linéaires, et des interactions plus fines entre variables. Or, sur ce dataset limité, moins de 900 observations après nettoyage, la complexité supplémentaire n'apporte pas beaucoup de valeur et peut même induire du bruit.

Retenons une règle clé en Machine Learning, un modèle simple, bien adapté aux données, peut surpasser des modèles plus sophistiqués.

Tous droits réservés à STUDI - Reproduction interdite

Voici le code de l'application si vous souhaitez avoir la structure de base, notez que la page 3 sera l'objet de notre prochaine étape.

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.svm import SVC
import xgboost as xgb

st.title(" 🚢 Simulation Titanic 🧊 ")

# --- Menu latéral ---
section = st.sidebar.radio(
    "Pages",
    ["Accueil", "Analyse", "Prédictions"]
)

# ------------------ " Page 1 " ------------------

# --- Contenu selon la section choisie ---
if section == "Accueil":
    st.header("Présentation")
    st.write('')
    st.write("Bienvenue 👋! Voici une présentation claire et concise du dataset Titanic et de son contexte :")
    st.write("Le Titanic est l'un des jeux de données les plus célèbres en data science et en apprentissage automatique. Il est utilisé dans la compétition Kaggle "Titanic: Machine Learning from Disaster", dont l'objectif est de prédire la probabilité de survie des passagers à partir de différentes informations disponibles lors du naufrage du 15 avril 1912.")
    st.subheader("📊 Contenu du dataset")
    st.write('')
    st.write("Les princiapes colonnes sont : ")
    st.write(" - PassengerId : Identifiant unique du passager")
    st.write("- Survived : Variable cible (0 = n'a pas survécu, 1 = a survécu) ")
    st.write("- Pclass : Classe du passager (1re, 2e ou 3e classe) ")
    st.write("- Name : Nom du passager")
    st.write("- Sex : Sexe")
    st.write("- Age : Âge (en années)")
    st.write("- SibSp : Nombre de frères/sœurs et conjoints à bord")
    st.write("- Parch : Nombre de parents et enfants à bord")
    st.write("- Ticket : Numéro de billet")
    st.write("-  Fare : Tarif du billet")
    st.write("- Cabin : Numéro de cabine")
    st.write("- Embarked : Port d'embarquement (C = Cherbourg, Q = Queenstown, S = Southampton)")
    st.write(" URL Kaggle : https://www.kaggle.com/competitions/titanic/overview")
    st.write('')
    st.header("🎯 Objectif du projet")
    st.write('')
    st.write('Le but est de prédire la survie d'un passager en fonction de ses caractéristiques. Cela se traduit par un problème de classification binaire :')
    st.write('- 1 (Survived) : le passager a survécu')
    st.write('- 0 (Not Survived) : le passager n'a pas survécu')

# ------------------ " Page 2 " ------------------

```


Tous droits réservés à STUDI - Reproduction interdite

```
elif section == "Analyse":
    st.header("Analyse exploratoire des données")

    # Onglets pour organiser l'analyse
    onglet1, onglet2, onglet3, onglet4 = st.tabs(["Etape 1", "Etape 2","Etape 3","Etape 4"])

    with onglet1:
        st.header("Etape 1 : Chargement et premier aperçu")
        st.subheader("🎯 Objectifs")
        st.write("- Charger correctement le dataset Titanic (version Kaggle).")
        st.write("- Vérifier ses dimensions (lignes × colonnes).")
        st.write("- Afficher les 5 premières lignes pour un aperçu.")
        st.write("- Vérifier les types de variables (int, float, object).")
        st.write("")
        st.write("Chargeons notre dataset depuis une url :")
        st.markdown("""```python
        url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
        df = pd.read_csv(url)
                    """)
        # Charger depuis une URL Kaggle (copie publique)
        url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
        df = pd.read_csv(url)

        # Dimensions
        st.subheader("Dimensions du dataset")
        st.write(f"{df.shape[0]} lignes × {df.shape[1]} colonnes")

        # Aperçu des 5 premières lignes
        st.subheader("Aperçu des données")
        st.dataframe(df.head())

        # Types des colonnes
        st.subheader("Types de colonnes")
        st.code(df.dtypes.to_string())
        st.write("")
        st.subheader("📝 Explications")
        st.write("- pd.read_csv(url) : charge directement les données depuis une URL publique")
        st.write("- df.shape : donne les dimensions du dataset.")
        st.write("- df.head() : montre les 5 premières lignes pour comprendre la structure.")
        st.write("- df.dtypes : permet de voir si les types sont cohérents (Age doit être numérique, Sex en objet, etc.).")


    with onglet2:
        st.header("Etape 2 : Structure et qualité des données ")
        st.subheader("🎯 Objectifs")
        st.write("- Identifier les colonnes avec des valeurs manquantes (NA).")
        st.write("- Vérifier s'il existe des doublons.")
        st.write("- Voir la cardinalité (nombre de valeurs uniques par colonne).")
        st.write("- Avoir une vision claire de la qualité globale des données.")

        # Vérification des valeurs manquantes
        st.subheader("Valeurs manquantes (%)")
        na_percent = df.isna().mean().round(3) * 100
        st.bar_chart(na_percent)

        # Heatmap des valeurs manquantes
        st.subheader("Heatmap des valeurs manquantes")
        fig, ax = plt.subplots(figsize=(10,5))
```

Tous droits réservés à STUDI - Reproduction interdite

```
        sns.heatmap(df.isnull(), cbar=False, cmap="mako", ax=ax)
        st.pyplot(fig)

        # Boxplots pour détecter les valeurs aberrantes
        st.subheader("Boxplots des variables numériques")

        num_cols = ["Age", "Fare"]
        for col in num_cols:
            fig, ax = plt.subplots(figsize=(6,3))
            sns.boxplot(x=df[col], ax=ax)
            ax.set_title(f"Boxplot de {col}")
            st.pyplot(fig)


    with onglet3 :
        st.header("Etape 3 : Préparation des données ")
        st.subheader("🎯 Objectifs")
        st.write("- Créer de nouvelles variables (features) qui peuvent améliorer la prédiction.")
        st.write("- Encoder les variables catégorielles pour les rendre numériques.")
        st.write("- Détecter et traiter les valeurs aberrantes (ex. Fare).")
        st.write("- Préparer un jeu de données final prêt pour l'entraînement.")

        # Gestion des valeurs manquantes
        df["Age"].fillna(df.groupby(["Sex", "Pclass"])["Age"].transform("median"), inplace=True)
        df["Embarked"].fillna(df["Embarked"].mode()[0], inplace=True)
        df["CabinKnown"] = df["Cabin"].notnull().astype(int)

        # Feature engineering
        df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
        df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
        df["Title"] = df["Name"].str.extract(r",\s*([^\.]+)\.")
        df["Deck"] = df["Cabin"].str[0].fillna("U")

        # Encodage
        encoder = OneHotEncoder(sparse_output=False, drop="first")
        encoded = encoder.fit_transform(df[["Sex", "Embarked", "Title", "Deck"]])
        encoded_cols = encoder.get_feature_names_out(["Sex", "Embarked", "Title", "Deck"])
        df_encoded = pd.DataFrame(encoded, columns=encoded_cols, index=df.index)

        # Normalisation
        scaler = StandardScaler()
        df[["Fare", "Age"]] = scaler.fit_transform(df[["Fare", "Age"]])

        # Dataset final
        final_df = pd.concat([df[["Survived","Pclass","Age","Fare","FamilySize","IsAlone","CabinKnown"]],
                              df_encoded], axis=1)
        st.write("  ")
        st.subheader("✅ Jeu de données final (prêt pour l'entraînement)")
        st.write("")
        st.dataframe(final_df.head(100))

    with onglet4:
        st.header("Etape 4 : Conception du modèle de Machine Learning ")
        st.subheader("🎯 Objectifs")
        st.write("- Séparer les données en jeu d'entraînement et jeu de test.")
        st.write("- Choisir un ou plusieurs algorithmes de ML adaptés.")
        st.write("- Entraîner un premier modèle de classification (baseline).")
        st.write("- Évaluer les performances avec des métriques adaptées (accuracy, précision, rappel, F1-score).")

```


Tous droits réservés à STUDI - Reproduction interdite

```
172         # Fonction de preprocessing
173         def preprocess_data(df):
174             df = df.drop(columns=["Name", "Ticket", "Cabin"], errors="ignore")
175             if "Embarked" in df.columns:
176                 df["Embarked"] = df["Embarked"].replace("U", "Unknown")
177             cat_cols = df.select_dtypes(include=["object"]).columns
178             if len(cat_cols) > 0:
179                 df = pd.get_dummies(df, columns=cat_cols, drop_first=True)
180             return df
181
182         df = preprocess_data(df)
183
184         # Séparation features / target
185         X = df.drop("Survived", axis=1)
186         y = df["Survived"]
187
188         X_train, X_test, y_train, y_test = train_test_split(
189             X, y, test_size=0.2, random_state=42, stratify=y
190         )
191
192         # Choix du modèle
193         st.subheader("Choix de l'algorithme")
194         algo = st.selectbox(
195             "Sélectionnez un modèle :",
196             ["Régression Logistique", "Random Forest (optimisé)", "Gradient
Boosting", "SVM", "XGBoost"]
197         )
198
199         # Régression Logistique
200         if algo == "Régression Logistique":
201             model = LogisticRegression(max_iter=1000)
202             model.fit(X_train, y_train)
203
204         # Random Forest Optimisé
205         elif algo == "Random Forest (optimisé)":
206             param_grid = {
207                 "n_estimators": [100, 200, 500],
208                 "max_depth": [None, 5, 10, 20],
209                 "min_samples_split": [2, 5, 10],
210                 "min_samples_leaf": [1, 2, 4]
211             }
212             grid = GridSearchCV(RandomForestClassifier(random_state=42),
213                                 param_grid,
214                                 cv=5,
215                                 scoring="accuracy",
216                                 n_jobs=-1)
217             grid.fit(X_train, y_train)
218             model = grid.best_estimator_
219             st.write("🌟 Meilleurs paramètres RF :", grid.best_params_)
220
221         # Gradient Boosting
222         elif algo == "Gradient Boosting":
223             model = GradientBoostingClassifier(n_estimators=200, learning_rate=0.05,
max_depth=3, random_state=42)
224             model.fit(X_train, y_train)
225
226         # SVM
227         elif algo == "SVM":
228             model = SVC(kernel="rbf", C=1, probability=True, random_state=42)
229             model.fit(X_train, y_train)
230
231         # XGBoost
```

Tous droits réservés à STUDI - Reproduction interdite

```
232         else:
233             model = xgb.XGBClassifier(
234                 n_estimators=200,
235                 learning_rate=0.05,
236                 max_depth=4,
237                 subsample=0.8,
238                 colsample_bytree=0.8,
239                 random_state=42,
240                 use_label_encoder=False,
241                 eval_metric="logloss"
242             )
243             model.fit(X_train, y_train)
244
245         # Évaluation
246         y_pred = model.predict(X_test)
247         acc = accuracy_score(y_test, y_pred)
248         st.metric("Accuracy", f"{acc:.2f}")
249
250         st.subheader("Rapport de classification")
251         st.text(classification_report(y_test, y_pred))
252
253         st.subheader("Matrice de confusion")
254         cm = confusion_matrix(y_test, y_pred)
255         fig, ax = plt.subplots()
256         sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=[0, 1],
yticklabels=[0, 1], ax=ax)
257         ax.set_xlabel("Prédiction")
258         ax.set_ylabel("Réalité")
259         st.pyplot(fig)
260
261
262
263
264 # ------------------ " Page 3 " ------------------
265
266 elif section == "Prédictions":
267     st.header("Paramètres de l'application")
268
269     # Onglets pour organiser les réglages
270     onglet1, onglet2 = st.tabs(["Options", "Avancé"])
271
272     with onglet1:
273         st.write("Réglages simples")
274         choix = st.selectbox("Choisis une option :", ["Option 1", "Option 2", "Option
3"])
275         st.write("Tu as choisi :", choix)
276
277     with onglet2:
278         st.write("Réglages avancés")
279         valeur = st.slider("Choisis une valeur :", 0, 100, 50)
280         st.write("Valeur sélectionnée :", valeur)
```


Tous droits réservés à STUDI - Reproduction interdite

Passons à notre dernière étape, elle consistera à saisir les caractéristiques d'un passager et nous aurons une prédiction en fonction.

Tous droits réservés à STUDI - Reproduction interdite

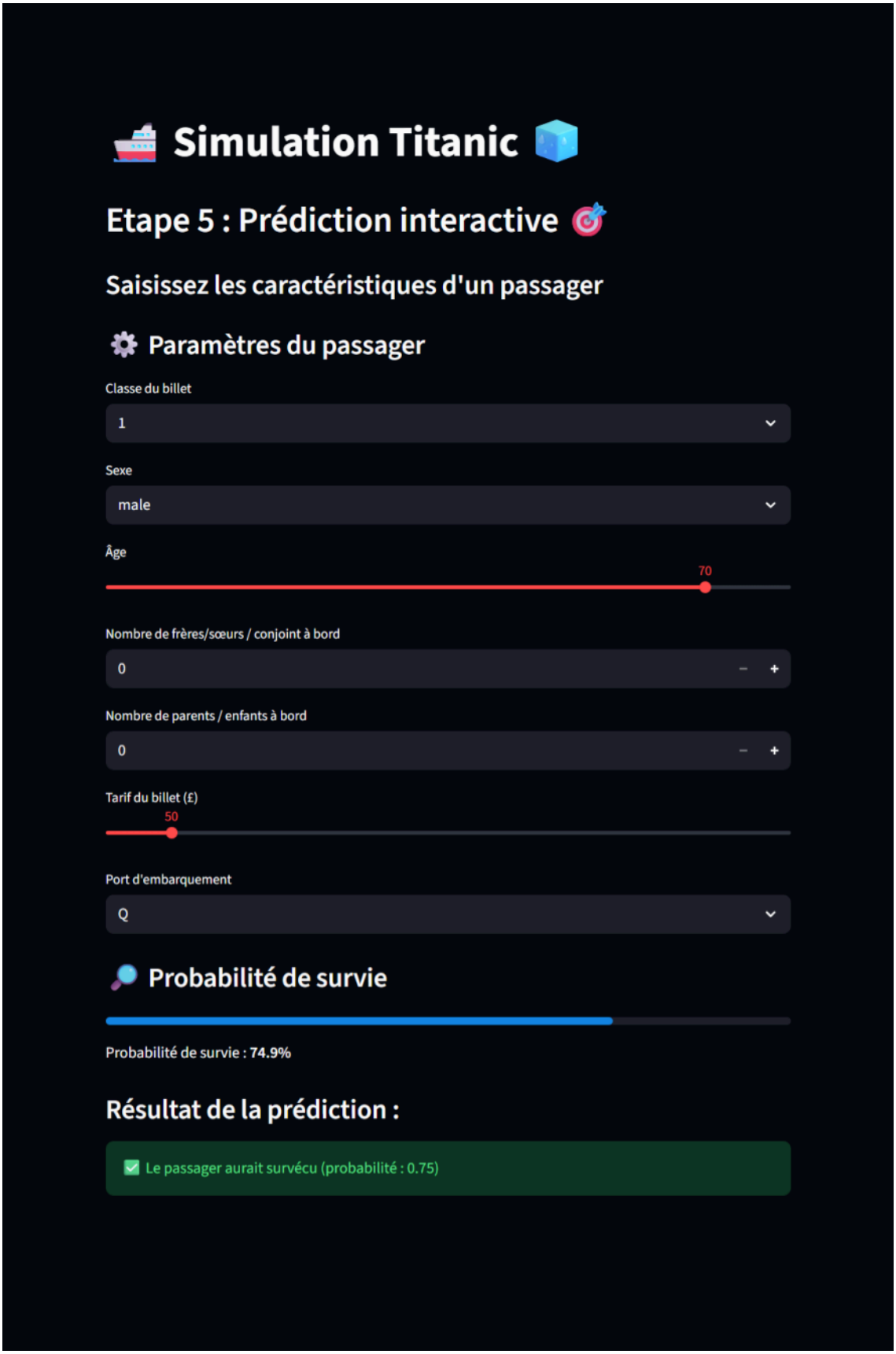
Voici le code source :

```
elif section == "Prédictions":
    from sklearn.linear_model import LogisticRegression

    st.header("Etape 5 : Prédiction interactive 🎯")
    st.subheader("Saisissez les caractéristiques d'un passager")

    # Charger le dataset brut
    url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
    df = pd.read_csv(url)

    # Prétraitement global (même logique que dans Analyse)
    def preprocess_data(df):
        df = df.drop(columns=["Name", "Ticket", "Cabin"], errors="ignore")
        if "Embarked" in df.columns:
            df["Embarked"] = df["Embarked"].fillna("S")
            df["Embarked"] = df["Embarked"].replace("U", "Unknown")
        if "Age" in df.columns:
            df["Age"].fillna(df.groupby(["Sex", "Pclass"])["Age"].transform("median"), inplace=True)
        cat_cols = df.select_dtypes(include=["object"]).columns
        if len(cat_cols) > 0:
            df = pd.get_dummies(df, columns=cat_cols, drop_first=True)
        return df

    # Jeu de données prétraité
    df = preprocess_data(df)

    # Features et cible
    X = df.drop("Survived", axis=1)
    y = df["Survived"]

    # Découpage en train/test
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # Entraînement d'une régression logistique
    model = LogisticRegression(max_iter=1000)
    model.fit(X_train, y_train)

    # Choix interactif des paramètres
    st.subheader("⚙️ Paramètres du passager")

    pclass = st.selectbox("Classe du billet", [1, 2, 3])
    sex = st.selectbox("Sexe", ["male", "female"])
    age = st.slider("Âge", 0, 80, 30)
    sibsp = st.number_input("Nombre de frères/sœurs / conjoint à bord", min_value=0, max_value=10, value=0)
    parch = st.number_input("Nombre de parents / enfants à bord", min_value=0, max_value=10, value=0)
    fare = st.slider("Tarif du billet (£)", 0, 520, 50)
    embarked = st.selectbox("Port d'embarquement", ["C", "Q", "S"])

    # DataFrame pour la prédiction
    input_data = pd.DataFrame({
        "Pclass": [pclass],
        "Sex": [sex],
        "Age": [age],
        "SibSp": [sibsp],
        "Parch": [parch],
        "Fare": [fare],
```


Tous droits réservés à STUDI - Reproduction interdite

```
59          "Embarked": [embarked]
60      })
61
62      # Appliquer le même prétraitement
63      input_data = preprocess_data(input_data)
64
65      # Harmoniser les colonnes
66      missing_cols = set(X_train.columns) - set(input_data.columns)
67      for col in missing_cols:
68          input_data[col] = 0
69      input_data = input_data[X_train.columns]
70
71      # Prédiction
72      prediction = model.predict(input_data)[0]
73      proba = model.predict_proba(input_data)[0][1]
74
75
76      # Jauge visuelle
77      st.subheader("🔎 Probabilité de survie")
78      # barre de progression
79      st.progress(int(proba * 100))
80      # Résultat
81      st.write(f"Probabilité de survie : **{proba*100:.1f}%**")
82      st.subheader("Résultat de la prédiction :")
83      if prediction == 1:
84          st.success(f"✅ Le passager aurait survécu (probabilité : {proba:.2f})")
85      else:
86          st.error(f"❌ Le passager n'aurait pas survécu (probabilité : {proba:.2f})")
```

Tous droits réservés à STUDI - Reproduction interdite



La page prédictions permet de tester le modèle de Machine Learning directement sur un cas personnalisé. L'utilisateur choisit les caractéristiques d'un passager du Titanic (classe, sexe, âge, nombre de proches à bord, prix du billet, port d'embarquement, etc.).

Le tout centralisé dans une application intuitive, et accessible à tous. Cette page rend le projet interactif et concret, elle met en valeur l'utilité pratique du Machine Learning en transformant des données historiques en un outil de simulation accessible à tous.

Vous pourrez retrouver l'intégralité du code dans votre espace.

Tous droits réservés à STUDI - Reproduction interdite

(cf. app.py)

```python
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.svm import SVC
import xgboost as xgb


st.title(" 🚢 Simulation Titanic 🧊 ")

# --- Menu latéral ---
section = st.sidebar.radio(
    "Pages",
    ["Accueil", "Analyse", "Prédictions"]
)

# ------------------ " Page 1 " ------------------

# --- Contenu selon la section choisie ---
if section == "Accueil":
    st.header("Présentation")
    st.write('')
    st.write("Bienvenue 👋! Voici une présentation claire et concise du dataset Titanic et de son contexte :")
    st.write("Le Titanic est l'un des jeux de données les plus célèbres en data science et en apprentissage automatique. Il est utilisé dans la compétition Kaggle "Titanic: Machine Learning from Disaster", dont l'objectif est de prédire la probabilité de survie des passagers à partir de différentes informations disponibles lors du naufrage du 15 avril 1912.")
    st.subheader("📊 Contenu du dataset")
    st.write('')
    st.write("Les princiapes colonnes sont : ")
    st.write(" - PassengerId : Identifiant unique du passager")
    st.write("- Survived : Variable cible (0 = n'a pas survécu, 1 = a survécu) ")
    st.write("- Pclass : Classe du passager (1re, 2e ou 3e classe) ")
    st.write("- Name : Nom du passager")
    st.write("- Sex : Sexe")
    st.write("- Age : Âge (en années)")
    st.write("- SibSp : Nombre de frères/sœurs et conjoints à bord")
    st.write("- Parch : Nombre de parents et enfants à bord")
    st.write("- Ticket : Numéro de billet")
    st.write("-  Fare : Tarif du billet")
    st.write("- Cabin : Numéro de cabine")
    st.write("- Embarked : Port d'embarquement (C = Cherbourg, Q = Queenstown, S = Southampton)")
    st.write(" URL Kaggle : https://www.kaggle.com/competitions/titanic/overview")
    st.write('')
    st.header("🎯 Objectif du projet")
    st.write('')
    st.write('Le but est de prédire la survie d'un passager en fonction de ses caractéristiques. Cela se traduit par un problème de classification binaire :')
    st.write('- 1 (Survived) : le passager a survécu')
    st.write('- 0 (Not Survived) : le passager n'a pas survécu')

# ------------------ " Page 2 " ------------------

```

Tous droits réservés à STUDI - Reproduction interdite

```
56 elif section == "Analyse":
57     st.header("Analyse exploratoire des données")
58 
59     # Onglets pour organiser l'analyse
60     onglet1, onglet2, onglet3, onglet4 = st.tabs(["Etape 1", "Etape 2","Etape
3","Etape 4"])
61 
62     with onglet1:
63         st.header("Etape 1 : Chargement et premier aperçu")
64         st.subheader("🎯 Objectifs")
65         st.write("- Charger correctement le dataset Titanic (version Kaggle).")
66         st.write("- Vérifier ses dimensions (lignes × colonnes).")
67         st.write("- Afficher les 5 premières lignes pour un aperçu.")
68         st.write("- Vérifier les types de variables (int, float, object).")
69         st.write("")
70         st.write("Chargeons notre dataset depuis une url :")
71         st.markdown("""```python
72         url =
"https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
73         df = pd.read_csv(url)
74                     """)
75         # Charger depuis une URL Kaggle (copie publique)
76         url =
"https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
77         df = pd.read_csv(url)
78 
79         # Dimensions
80         st.subheader("Dimensions du dataset")
81         st.write(f"{df.shape[0]} lignes × {df.shape[1]} colonnes")
82 
83         # Aperçu des 5 premières lignes
84         st.subheader("Aperçu des données")
85         st.dataframe(df.head())
86 
87         # Types des colonnes
88         st.subheader("Types de colonnes")
89         st.code(df.dtypes.to_string())
90         st.write("")
91         st.subheader("📝 Explications")
92         st.write("- pd.read_csv(url) : charge directement les données depuis une URL
publique")
93         st.write("- df.shape : donne les dimensions du dataset.")
94         st.write("- df.head() : montre les 5 premières lignes pour comprendre la
structure.")
95         st.write("- df.dtypes : permet de voir si les types sont cohérents (Age doit
être numérique, Sex en objet, etc.).")
96 
97 
98     with onglet2:
99         st.header("Etape 2 : Structure et qualité des données ")
100        st.subheader("🎯 Objectifs")
101        st.write("- Identifier les colonnes avec des valeurs manquantes (NA).")
102        st.write("- Vérifier s'il existe des doublons.")
103        st.write("- Voir la cardinalité (nombre de valeurs uniques par colonne).")
104        st.write("- Avoir une vision claire de la qualité globale des données.")
105 
106        # Vérification des valeurs manquantes
107        st.subheader("Valeurs manquantes (%)")
108        na_percent = df.isna().mean().round(3) * 100
109        st.bar_chart(na_percent)
110 
111        # Heatmap des valeurs manquantes
112        st.subheader("Heatmap des valeurs manquantes")
113        fig, ax = plt.subplots(figsize=(10,5))
```


Tous droits réservés à STUDI - Reproduction interdite

```
        sns.heatmap(df.isnull(), cbar=False, cmap="mako", ax=ax)
        st.pyplot(fig)

        # Boxplots pour détecter les valeurs aberrantes
        st.subheader("Boxplots des variables numériques")

        num_cols = ["Age", "Fare"]
        for col in num_cols:
            fig, ax = plt.subplots(figsize=(6,3))
            sns.boxplot(x=df[col], ax=ax)
            ax.set_title(f"Boxplot de {col}")
            st.pyplot(fig)


    with onglet3 :
        st.header("Etape 3 : Préparation des données ")
        st.subheader("🎯 Objectifs")
        st.write("- Créer de nouvelles variables (features) qui peuvent améliorer la prédiction.")
        st.write("- Encoder les variables catégorielles pour les rendre numériques.")
        st.write("- Détecter et traiter les valeurs aberrantes (ex. Fare).")
        st.write("- Préparer un jeu de données final prêt pour l'entraînement.")

        # Gestion des valeurs manquantes
        df["Age"].fillna(df.groupby(["Sex", "Pclass"])["Age"].transform("median"), inplace=True)
        df["Embarked"].fillna(df["Embarked"].mode()[0], inplace=True)
        df["CabinKnown"] = df["Cabin"].notnull().astype(int)

        # Feature engineering
        df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
        df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
        df["Title"] = df["Name"].str.extract(r",\s*([^\.]+)\.")
        df["Deck"] = df["Cabin"].str[0].fillna("U")

        # Encodage
        encoder = OneHotEncoder(sparse_output=False, drop="first")
        encoded = encoder.fit_transform(df[["Sex", "Embarked", "Title", "Deck"]])
        encoded_cols = encoder.get_feature_names_out(["Sex", "Embarked", "Title", "Deck"])
        df_encoded = pd.DataFrame(encoded, columns=encoded_cols, index=df.index)

        # Normalisation
        scaler = StandardScaler()
        df[["Fare", "Age"]] = scaler.fit_transform(df[["Fare", "Age"]])

        # Dataset final
        final_df = pd.concat([df[["Survived","Pclass","Age","Fare","FamilySize","IsAlone","CabinKnown"]],
                              df_encoded], axis=1)
        st.write("  ")
        st.subheader("✅ Jeu de données final (prêt pour l'entraînement)")
        st.write("")
        st.dataframe(final_df.head(100))

    with onglet4:
        st.header("Etape 4 : Conception du modèle de Machine Learning ")
        st.subheader("🎯 Objectifs")
        st.write("- Séparer les données en jeu d'entraînement et jeu de test.")
        st.write("- Choisir un ou plusieurs algorithmes de ML adaptés.")
        st.write("- Entraîner un premier modèle de classification (baseline).")
        st.write("- Évaluer les performances avec des métriques adaptées (accuracy, précision, rappel, F1-score).")

```

Tous droits réservés à STUDI - Reproduction interdite

```
173         # Fonction de preprocessing
174         def preprocess_data(df):
175             df = df.drop(columns=["Name", "Ticket", "Cabin"], errors="ignore")
176             if "Embarked" in df.columns:
177                 df["Embarked"] = df["Embarked"].replace("U", "Unknown")
178             cat_cols = df.select_dtypes(include=["object"]).columns
179             if len(cat_cols) > 0:
180                 df = pd.get_dummies(df, columns=cat_cols, drop_first=True)
181             return df
182 
183         df = preprocess_data(df)
184 
185         # Séparation features / target
186         X = df.drop("Survived", axis=1)
187         y = df["Survived"]
188 
189         X_train, X_test, y_train, y_test = train_test_split(
190             X, y, test_size=0.2, random_state=42, stratify=y
191         )
192 
193         # Choix du modèle
194         st.subheader("Choix de l'algorithme")
195         algo = st.selectbox(
196             "Sélectionnez un modèle :",
197             ["Régression Logistique", "Random Forest (optimisé)", "Gradient
Boosting", "SVM", "XGBoost"]
198         )
199 
200         # Régression Logistique
201         if algo == "Régression Logistique":
202             model = LogisticRegression(max_iter=1000)
203             model.fit(X_train, y_train)
204 
205         # Random Forest Optimisé
206         elif algo == "Random Forest (optimisé)":
207             param_grid = {
208                 "n_estimators": [100, 200, 500],
209                 "max_depth": [None, 5, 10, 20],
210                 "min_samples_split": [2, 5, 10],
211                 "min_samples_leaf": [1, 2, 4]
212             }
213             grid = GridSearchCV(RandomForestClassifier(random_state=42),
214                                 param_grid,
215                                 cv=5,
216                                 scoring="accuracy",
217                                 n_jobs=-1)
218             grid.fit(X_train, y_train)
219             model = grid.best_estimator_
220             st.write("🌟 Meilleurs paramètres RF :", grid.best_params_)
221 
222         # Gradient Boosting
223         elif algo == "Gradient Boosting":
224             model = GradientBoostingClassifier(n_estimators=200, learning_rate=0.05,
max_depth=3, random_state=42)
225             model.fit(X_train, y_train)
226 
227         # SVM
228         elif algo == "SVM":
229             model = SVC(kernel="rbf", C=1, probability=True, random_state=42)
230             model.fit(X_train, y_train)
231 
232         # XGBoost
```


Tous droits réservés à STUDI - Reproduction interdite

```
        else:
            model = xgb.XGBClassifier(
                n_estimators=200,
                learning_rate=0.05,
                max_depth=4,
                subsample=0.8,
                colsample_bytree=0.8,
                random_state=42,
                use_label_encoder=False,
                eval_metric="logloss"
            )
            model.fit(X_train, y_train)

        # Évaluation
        y_pred = model.predict(X_test)
        acc = accuracy_score(y_test, y_pred)
        st.metric("Accuracy", f"{acc:.2f}")

        st.subheader("Rapport de classification")
        st.text(classification_report(y_test, y_pred))

        st.subheader("Matrice de confusion")
        cm = confusion_matrix(y_test, y_pred)
        fig, ax = plt.subplots()
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=[0, 1], yticklabels=[0, 1], ax=ax)
        ax.set_xlabel("Prédiction")
        ax.set_ylabel("Réalité")
        st.pyplot(fig)

elif section == "Prédictions":
    from sklearn.linear_model import LogisticRegression

    st.header("Etape 5 : Prédiction interactive 🎯")
    st.subheader("Saisissez les caractéristiques d'un passager")

    # Charger le dataset brut
    url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
    df = pd.read_csv(url)

    # Prétraitement global (même logique que dans Analyse)
    def preprocess_data(df):
        df = df.drop(columns=["Name", "Ticket", "Cabin"], errors="ignore")
        if "Embarked" in df.columns:
            df["Embarked"] = df["Embarked"].fillna("S")
            df["Embarked"] = df["Embarked"].replace("U", "Unknown")
        if "Age" in df.columns:
            df["Age"].fillna(df.groupby(["Sex", "Pclass"])["Age"].transform("median"), inplace=True)
        cat_cols = df.select_dtypes(include=["object"]).columns
        if len(cat_cols) > 0:
            df = pd.get_dummies(df, columns=cat_cols, drop_first=True)
        return df

    # Jeu de données prétraité
    df = preprocess_data(df)

    # Features et cible
    X = df.drop("Survived", axis=1)
    y = df["Survived"]

    # Découpage en train/test
```

Tous droits réservés à STUDI - Reproduction interdite

```
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # Entraînement d'une régression logistique
    model = LogisticRegression(max_iter=1000)
    model.fit(X_train, y_train)

    # Choix interactif des paramètres
    st.subheader("⚙️ Paramètres du passager")

    pclass = st.selectbox("Classe du billet", [1, 2, 3])
    sex = st.selectbox("Sexe", ["male", "female"])
    age = st.slider("Âge", 0, 80, 30)
    sibsp = st.number_input("Nombre de frères/sœurs / conjoint à bord", min_value=0, max_value=10, value=0)
    parch = st.number_input("Nombre de parents / enfants à bord", min_value=0, max_value=10, value=0)
    fare = st.slider("Tarif du billet (£)", 0, 520, 50)
    embarked = st.selectbox("Port d'embarquement", ["C", "Q", "S"])

    # DataFrame pour la prédiction
    input_data = pd.DataFrame({
        "Pclass": [pclass],
        "Sex": [sex],
        "Age": [age],
        "SibSp": [sibsp],
        "Parch": [parch],
        "Fare": [fare],
        "Embarked": [embarked]
    })

    # Appliquer le même prétraitement
    input_data = preprocess_data(input_data)

    # Harmoniser les colonnes
    missing_cols = set(X_train.columns) - set(input_data.columns)
    for col in missing_cols:
        input_data[col] = 0
    input_data = input_data[X_train.columns]

    # Prédiction
    prediction = model.predict(input_data)[0]
    proba = model.predict_proba(input_data)[0][1]


    # Jauge visuelle
    st.subheader("🔎 Probabilité de survie")
    # barre de progression
    st.progress(int(proba * 100))
    # Résultat
    st.write(f"Probabilité de survie : **{proba*100:.1f}%**")
    st.subheader("Résultat de la prédiction :")
    if prediction == 1:
        st.success(f"✅ Le passager aurait survécu (probabilité : {proba:.2f})")
    else:
        st.error(f"❌ Le passager n'aurait pas survécu (probabilité : {proba:.2f})")
```


Tous droits réservés à STUDI - Reproduction interdite

III Packaging, reproductibilité et maintenance

La gestion du cycle de vie d'un modèle de Machine Learning ne s'arrête pas à son entraînement initial. Sans système de versioning, il devient rapidement impossible de retracer l'historique, quelle version du modèle est actuellement en production, avec quelles données il a été entraîné, et dans quel état il se trouvait au moment du déploiement. Dans notre cas il est difficile d'ajouter des données à notre dataset car il concerne un événement historique du siècle passé. Nous pourrions faire à nouveau du feature engineering et essayer de garder les meilleurs résultats sur les différents algorithmes. Ce qui nous amènerait à suivre les différentes versions et de s'assurer de la traçabilité et la reproductibilité de nos travaux.

De la même manière, l'intégration de tests automatisés à chaque étape du projet joue un rôle clé dans l'assurance qualité, ils permettent de détecter rapidement d'éventuelles régressions et de garantir que les performances du modèle ne se dégradent pas au fil des évolutions, et des modifications que nous pourrions apporter.

La documentation est un autre pilier trop souvent sous-estimé. Elle facilite la compréhension et la reprise du projet par d'autres équipes, ou même par soi-même après plusieurs mois. Y consigner les choix techniques, les décisions de conception et les résultats obtenus permet de garder une vision claire du chemin parcouru. C'est pour cela que beaucoup de personnes considèrent le simple fait de créer une application Streamlit comme le premier pas vers une documentation collaborative. Nous aurions pu ajouter une page/onglet logs/documentation, pour « *monitorer et décrire* » nos avancées et choix techniques.

La dernière étape manquante est le déploiement de notre application, étape clé pour rendre le projet accessible aux utilisateur finaux ou parties prenantes. Dans notre cas, notre application Streamlit peut être hébergée sur Heroku, Docker ou Streamlit Cloud. Il est essentiel d'accompagner ce déploiement d'un système de versioning non seulement pour le code, mais aussi pour les modèles intégrés. Cette pratique évitant toute confusion en production et permet si nécessaire de revenir rapidement à une version antérieure stable.

Tous droits réservés à STUDI - Reproduction interdite

IV L'essentiel

Dans ce cours, nous avons vu que l'analyse exploratoire constitue une étape essentielle avant toute modélisation. Elle permet d'évaluer la qualité des données, de repérer les valeurs manquantes, les doublons et surtout la cardinalité des colonnes, ainsi que les éventuelles anomalies. Cette phase conditionne directement la fiabilité des résultats et la pertinence des modèles que nous produirons et utiliserons.

La préparation des données est tout aussi importante, car les données brutes ne sont presque jamais exploitables en l'état. L'imputation de valeurs manquantes, l'encodage des variables catégorielles et la normalisation des variables numériques permettent de rendre le jeu de données cohérent et uniforme. Le feature engineering que nous avons réalisé illustre le rôle de la créativité et de la compréhension métier dans la data science. En enrichissant les données avec des variables comme la taille de la famille, le fait d'être seul ou accompagné, ou encore les titres extraits des noms, permettent de capter des signaux cachés qui peuvent améliorer significativement la qualité prédictive des modèles et leurs choix.

En entreprise, ces pratiques se traduisent par des modèles plus robustes et adaptés à la réalité opérationnelle, toutefois attention car parfois en faisant du feature engineering certaines données peuvent disparaître car jugée non pertinentes seules, et d'autres peuvent apparaître. C'est assez rare, mais imaginons que dans notre dataset le groupe sanguin, ou les habitudes alimentaires aient été demandées aux passagers. Est-ce que cela aurait changé les chances de survie des passagers ? Très certainement non. Mais dans des usages en médecine, l'origine ethnique, l'alimentation peut expliquer certaines comorbidités.

En bilan de cette leçon, la réussite d'un projet de Machine Learning repose avant tout sur la rigueur du traitement des données. Les modèles ne valent que par la qualité des données qui les alimentent, et c'est dans cette phrase de préparation que se construit la solidité d'un projet d'intelligence artificielle.

Tous droits réservés à STUDI - Reproduction interdite

V Synthèse

A. Flashcards

Déploiement IA

Rendre un modèle accessible, maintenable et suivi dans le temps.

Traçabilité modèle

Suivi des versions, données et paramètres d'entraînement du modèle.

Streamlit

Framework Python pour créer facilement des applications web interactives.

Feature engineering

Création de nouvelles variables pour améliorer la prédiction.

Versioning

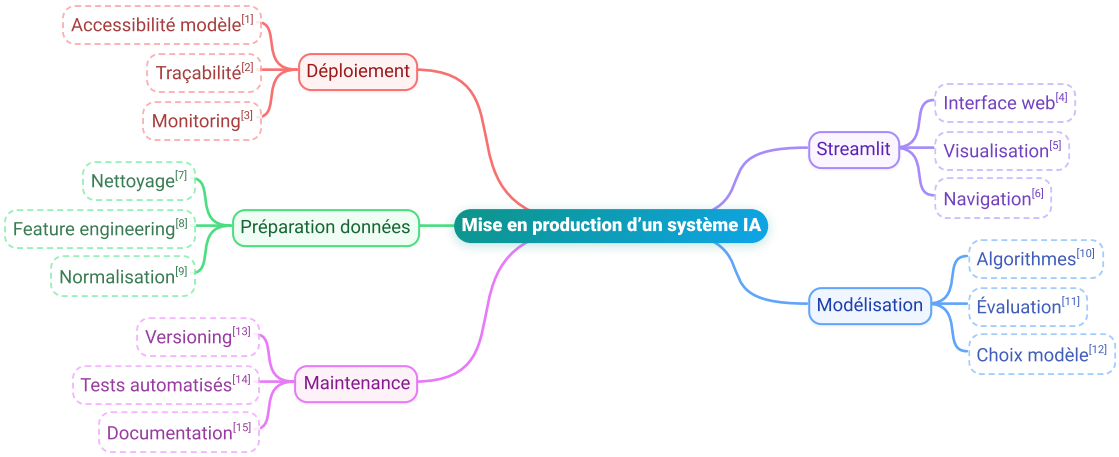
Gestion des différentes versions de modèles et données pour reproductibilité.

La théorie en Bref

Fondamental

Le **déploiement** d'un système d'**IA** consiste à rendre un modèle accessible et maintenable dans le temps. La **traçabilité** est essentielle pour suivre les versions du modèle et détecter son obsolescence. La **reproductibilité** nécessite un **versioning** rigoureux des modèles et des données, souvent via des outils comme Mlflow ou DVC. Streamlit permet de créer rapidement des **interfaces web interactives** pour rendre les modèles utilisables par des utilisateurs non techniques. La réussite d'un projet IA dépend de la qualité des données, de la rigueur dans la préparation, et de la maintenance continue du modèle.

Tous droits réservés à STUDI - Reproduction interdite



1. Modèle doit être accessible et compréhensible
2. Suivi versions, paramètres, données d'entraînement
3. Surveillance erreurs pour réentraînement
4. Création rapide d'applications interactives
5. Cartes, graphiques, formulaires utilisateurs
6. Onglets, sidebar, menus pour structurer
7. Imputation valeurs manquantes, suppression
8. Création variables, encodage catégoriel
9. Mise à l'échelle variables numériques
10. Régression logistique, Random Forest, XGBoost
11. Accuracy, précision, rappel, matrice confusion
12. Simplicité parfois préférable
13. Mlflow, DVC pour suivi modèles et données
14. Détection régressions et anomalies
15. Traçabilité choix techniques et évolutions

Mise en production d'un système IA

Les conseils pro

Conseil

Comment assurer la traçabilité d'un modèle IA en production ?

- Utiliser un système de versioning pour modèles et données.
- Documenter les paramètres et jeux de données utilisés.
- Mettre en place un monitoring régulier des performances.
- Archiver les logs et résultats des entraînements.

Comment concevoir une interface utilisateur avec Streamlit ?

- Structurer l'application avec onglets et sidebar.
- Intégrer des visualisations interactives adaptées aux données.
- Simplifier les interactions pour utilisateurs non techniques.
- Tester régulièrement les mises à jour de l'interface.

Comment préparer les données pour un modèle prédictif ?

Tous droits réservés à STUDI - Reproduction interdite

- Imputer ou supprimer les valeurs manquantes de façon cohérente.
- Créer des variables pertinentes via le feature engineering.
- Encoder les variables catégorielles correctement.
- Normaliser les variables numériques pour homogénéiser les échelles.

Comment choisir et évaluer un modèle de Machine Learning ?

- Tester plusieurs algorithmes adaptés au problème.
- Utiliser des métriques pertinentes (accuracy, rappel, F1-score).
- Privilégier un modèle simple si les performances sont similaires.
- Analyser la matrice de confusion pour comprendre erreurs.

Comment maintenir un modèle IA en production ?

- Mettre en place des tests automatisés pour détecter dérives.
- Surveiller les performances et déclencher réentraînement si besoin.
- Documenter les évolutions et choix techniques.
- Garder un historique clair des versions déployées.

Tous droits réservés à STUDI - Reproduction interdite

VI Auto-évaluation

A. Test

Exercice 1 : Quiz

[solution n°1 p. 47]

Question 1

Quelle est la première étape avant d'entraîner un modèle de Machine Learning ?

- Déployer le modèle en production
- Normaliser toutes les colonnes
- Réaliser une analyse exploratoire des données
- Supprimer toutes les valeurs manquantes

Question 2

Quelle est la variable cible dans le dataset Titanic ?

- Pclass
- Survived
- Fare
- Age

Question 3

Pourquoi il est important de traiter les valeurs manquantes ?

- Pour réduire la taille du dataset
- Parce que les algorithmes ne peuvent pas gérer les NaN
- Pour cacher les colonnes inutiles
- Parce que les doublons deviennent invisibles

Question 4

Quel est l'intérêt du feature engineering ?

Tous droits réservés à STUDI - Reproduction interdite

Supprimer les données inutiles

Créer de nouvelles variables plus pertinentes

Réduire la taille du dataset

Éviter l'overfitting

Question 5

Quelle est la technique la plus utilisée pour encoder les variables catégorielles ?

StandardScaler

One-Hot Encoding

PCA

Normalisation Min-Max

Question 6

Pourquoi normaliser les variables numériques comme Fare et Age ?

Pour réduire le nombre de colonnes

Pour uniformiser les échelles et éviter qu'une variable domine l'autre

Pour supprimer les doublons

Pour encoder les variables catégorielles

Question 7

Pourquoi la Random Forest peut être moins performante que la régression logistique sur le dataset Titanic ?

Parce qu'elle ne gère pas les valeurs catégorielles

Parce que le dataset est petit et linéaire

Parce que les forêts nécessitent plus de données pour converger

Les deux réponses B et C

Tous droits réservés à STUDI - Reproduction interdite

Question 8

Quelle est l'utilité principale d'une heatmap des valeurs manquantes ?

- Remplir automatiquement les NaN
- Visualiser où se situent les trous dans les données
- Supprimer les lignes incomplètes
- Réduire le nombre de variables

Question 9

Quelle libraire Python est utilisée pour créer une application web interactive dans ce projet ?

- Flask
- Django
- Streamlit
- Dash

Question 10

Quelle est le message clé de la leçon ?

- Un modèle complexe est toujours meilleur
- La préparation des données est plus importante que le choix de l'algorithme
- Le dataset Titanic est parfait et n'a pas besoin de nettoyage
- Streamlit remplace les étapes de prétraitement

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 44]

Question 1

Quelle est la première étape avant d'entraîner un modèle de Machine Learning ?

- Déployer le modèle en production
- Normaliser toutes les colonnes
- ✓ Réaliser une analyse exploratoire des données
- Supprimer toutes les valeurs manquantes

L'exploration des données est indispensable pour comprendre la qualité et la structure du dataset avant tout nettoyage et modélisation.

Question 2

Quelle est la variable cible dans le dataset Titanic ?

- Pclass
- ✓ Survived
- Fare
- Age

La variable Survived (0 = mort , 1 = survivant) est la cible de classification.

Question 3

Pourquoi il est important de traiter les valeurs manquantes ?

- Pour réduire la taille du dataset
- ✓ Parce que les algorithmes ne peuvent pas gérer les NaN

Tous droits réservés à STUDI - Reproduction interdite

Pour cacher les colonnes inutiles

Parce que les doublons deviennent invisibles

La plupart des algorithmes classiques de Machine Learning ne tolèrent pas les valeurs manquantes, elles doivent être imputées ou supprimées.

Question 4

Quel est l'intérêt du feature engineering ?

Supprimer les données inutiles

✓ Créer de nouvelles variables plus pertinentes

Réduire la taille du dataset

Éviter l'overfitting

Le feature engineering permet de capturer des signaux cachés en créant des variables utiles comme FamilySize, IsAlone ou Title

Question 5

Quelle est la technique la plus utilisée pour encoder les variables catégorielles ?

StandardScaler

✓ One-Hot Encoding

PCA

Normalisation Min-Max

Le One-Hot Encoding transforme les catégories (Sex, Embarked, Title, Deck) en colonnes binaires exploitables par les modèles.

Question 6

Pourquoi normaliser les variables numériques comme Fare et Age ?

Pour réduire le nombre de colonnes

Tous droits réservés à STUDI - Reproduction interdite

✓ Pour uniformiser les échelles et éviter qu'une variable domine l'autre

Pour supprimer les doublons

Pour encoder les variables catégorielles

La normalisation permet de mettre toutes les variables sur une échelle comparable, ce qui est crucial pour certains algorithmes.

Question 7

Pourquoi la Random Forest peut être moins performante que la régression logistique sur le dataset Titanic ?

Parce qu'elle ne gère pas les valeurs catégorielles

Parce que le dataset est petit et linéaire

Parce que les forêts nécessitent plus de données pour converger

✓ Les deux réponses B et C

La Random Forest est puissante sur des datasets complexes, mais ici la simplicité du problème rend la régression logistique plus efficace.

Question 8

Quelle est l'utilité principale d'une heatmap des valeurs manquantes ?

Remplir automatiquement les NaN

✓ Visualiser où se situent les trous dans les données

Supprimer les lignes incomplètes

Réduire le nombre de variables

La heatmap met en évidence la distribution des données manquantes par colonne et par ligne.

Question 9

Quelle libraire Python est utilisée pour créer une application web interactive dans ce projet ?

Tous droits réservés à STUDI - Reproduction interdite

Flask

Django

✓ Streamlit

Dash

Streamlit permet de construire rapidement des interfaces interactives pour data science et Machine Learning.

Question 10

Quelle est le message clé de la leçon ?

Un modèle complexe est toujours meilleur

✓ La préparation des données est plus importante que le choix de l'algorithme

Le dataset Titanic est parfait et n'a pas besoin de nettoyage

Streamlit remplace les étapes de prétraitement

La rigueur dans l'exploration et la préparation des données détermine la qualité du projet IA, plus que la complexité du modèle choisi.

Tous droits réservés à STUDI - Reproduction interdite